AI

April 23, 2026 at 12:44 PM

Gemini Conversation 1e6b9635

25 conversation turns

Turn 1

User

```
import torch
import torchvision
import torchvision.transforms as transforms
import numpy as np
import cv2
from PIL import Image
import matplotlib.pyplot as plt
# ==========================================
# 1. 自定义干扰变换 (Noise, Occlusion, Stylization)
# ==========================================
class AddSaltPepperNoise(object):
"""添加 50% 椒盐噪声"""
def __init__(self, prob=0.5):
self.prob = prob
def __call__(self, img):
img_arr = np.array(img)
row, col, ch = img_arr.shape
s_vs_p = 0.5
amount = self.prob
out = np.copy(img_arr)
# 添加盐噪声 (Salt)
num_salt = np.ceil(amount * img_arr.size * s_vs_p)
coords = [np.random.randint(0, i - 1, int(num_salt)) for i in img_arr.shape]
out[tuple(coords)] = 255
# 添加椒噪声 (Pepper)
num_pepper = np.ceil(amount * img_arr.size * (1. - s_vs_p))
coords = [np.random.randint(0, i - 1, int(num_pepper)) for i in img_arr.shape]
out[tuple(coords)] = 0
```

```
return Image.fromarray(out)
class SketchStylization(object):
def __call__(self, img):
img_arr = np.array(img)
gray_sketch, color_sketch = cv2.pencilSketch(img_arr, sigma_s=60, sigma_r=0.07, shade_factor=0.05)
sketch_rgb = cv2.cvtColor(gray_sketch, cv2.COLOR_GRAY2RGB)
return Image.fromarray(sketch_rgb)
# ==========================================
# 2. 数据处理管道与数据加载器 (DataLoader)
# ==========================================
def get_cifar10_dataloaders(batch_size=64):
"""获取基础以及受干扰的 CIFAR-10 数据加载器"""
base_transform = transforms.Compose([
transforms.ToTensor(),
transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
])
# 1. 理想场景 (Ideal Scenario)
train_dataset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True, transform=base_transform)
test_clean = torchvision.datasets.CIFAR10(root='./data', train=False, download=True, transform=base_transform)
# 2. 50% 椒盐噪声场景
transform_noise = transforms.Compose([
AddSaltPepperNoise(prob=0.5),
transforms.ToTensor(),
transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
])
test_noise = torchvision.datasets.CIFAR10(root='./data', train=False, download=True, transform=transform_noise)
# 3. 30% 区域遮挡场景
transform_occlusion = transforms.Compose([
```

```
transforms.ToTensor(),
transforms.RandomErasing(p=1.0, scale=(0.3, 0.3), ratio=(1.0, 1.0), value=0), # 严格 30% 面积
transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
])
test_occlusion = torchvision.datasets.CIFAR10(root='./data', train=False, download=True, transform=transform_occlusion)
# 4. 素描风格化场景
transform_sketch = transforms.Compose([
SketchStylization(),
transforms.ToTensor(),
transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
])
test_sketch = torchvision.datasets.CIFAR10(root='./data', train=False, download=True, transform=transform_sketch)
dataloaders = {
'train': torch.utils.data.DataLoader(train_dataset, batch_size=batch_size, shuffle=True, num_workers=0),
'test_clean': torch.utils.data.DataLoader(test_clean, batch_size=batch_size, shuffle=False, num_workers=0),
'test_noise': torch.utils.data.DataLoader(test_noise, batch_size=batch_size, shuffle=False, num_workers=0),
'test_occlusion': torch.utils.data.DataLoader(test_occlusion, batch_size=batch_size, shuffle=False, num_workers=0),
'test_sketch': torch.utils.data.DataLoader(test_sketch, batch_size=batch_size, shuffle=False, num_workers=0),
}
return dataloaders
# ==========================================
# 3. 核心可视化模块
# ==========================================
def visualize_transformations():
print("⏳ 正在生成干扰数据的可视化对比矩阵图...")
# 获取最原始的数据集（不进行 Normalize，保持可展示的 PIL 格式）
```

```
dataset_raw = torchvision.datasets.CIFAR10(root='./data', train=False, download=True)
classes = dataset_raw.classes
noise_transform = AddSaltPepperNoise(prob=0.5)
sketch_transform = SketchStylization()
transform_to_tensor = transforms.ToTensor()
erasing_transform = transforms.RandomErasing(p=1.0, scale=(0.3, 0.3), ratio=(1.0, 1.0), value=0)
indices = [3, 4, 5, 14]
fig, axes = plt.subplots(4, 4, figsize=(10, 10), dpi=200)
fig.suptitle('CIFAR-10: Ideal vs Non-Ideal Scenarios', fontsize=18, fontweight='bold', y=0.95)
col_titles = ['Original (Clean)', 'Salt & Pepper Noise', 'Spatial Occlusion', 'Sketch Stylization']
for i, idx in enumerate(indices):
img_pil, label = dataset_raw[idx]
# 1. 原图
img_clean = np.array(img_pil)
# 2. 椒盐噪声
img_noise = np.array(noise_transform(img_pil))
# 3. 遮挡 (RandomErasing 需要 Tensor 输入)
img_tensor = transform_to_tensor(img_pil)
img_erased = erasing_transform(img_tensor)
img_occlusion = img_erased.permute(1, 2, 0).numpy() # 转回 (H,W,C) 用于显示
# 4. 素描化
img_sketch = np.array(sketch_transform(img_pil))
images = [img_clean, img_noise, img_occlusion, img_sketch]
for j in range(4):
ax = axes[i, j]
if j == 2:
ax.imshow(images[j])
else:
ax.imshow(images[j].astype(np.uint8))
ax.axis('off')
if i == 0:
```

ax.set_title(col_titles[j], fontsize=12, fontweight='bold', pad=10)

if j == 0:

ax.text(-0.2, 0.5, classes[label].capitalize(),

transform=ax.transAxes, fontsize=14, fontweight='bold',

va='center', ha='right', rotation=90)

plt.tight_layout()

plt.subplots_adjust(top=0.88, left=0.1)

save_path = 'cifar10_corruptions_preview.png'

plt.savefig(save_path, bbox_inches='tight', transparent=False)

print(f"✅ 可视化矩阵已生成并安全保存至: {save_path}")

plt.show()

==========

4. 主运行入口

==========

if __name__ == "__main__":

visualize_transformations()

print("\n" + "="*50)

print("正在准备带有非理想场景干扰的 CIFAR-10 数据加载器...")

ldataloaders = get_cifar10_dataloaders(batch_size=128)

print("数据加载器准备完毕！包含以下测试集：")

for key in loaders.keys():

print(f" - {key.ljust(15)}: {len(loaders[key].dataset)} 张图像")不修改代码，将注释和输出转换成英文

🤖 Assistant

Here is your code with all comments and output statements translated into English, while strictly preserving the original logic and structure:

```
 import torch
import torchvision
import torchvision.transforms as transforms
import numpy as np
import cv2
from PIL import Image
import matplotlib.pyplot as plt
```

```
# ==========================================
# 1. Custom Corruption Transformations (Noise, Occlusion, Stylization)
# ==========================================

class AddSaltPepperNoise(object):
    """Add 50% Salt and Pepper Noise"""
    def __init__(self, prob=0.5):
        self.prob = prob

    def __call__(self, img):
        img_arr = np.array(img)
        row, col, ch = img_arr.shape
        s_vs_p = 0.5
        amount = self.prob
        out = np.copy(img_arr)

        # Add Salt Noise
        num_salt = np.ceil(amount * img_arr.size * s_vs_p)
        coords = [np.random.randint(0, i - 1, int(num_salt)) for i in
img_arr.shape]
        out[tuple(coords)] = 255

        # Add Pepper Noise
        num_pepper = np.ceil(amount * img_arr.size * (1. - s_vs_p))
        coords = [np.random.randint(0, i - 1, int(num_pepper)) for i in
img_arr.shape]
        out[tuple(coords)] = 0

        return Image.fromarray(out)

class SketchStylization(object):
    def __call__(self, img):
        img_arr = np.array(img)
        gray_sketch, color_sketch = cv2.pencilSketch(img_arr, sigma_s=60,
sigma_r=0.07, shade_factor=0.05)

        sketch_rgb = cv2.cvtColor(gray_sketch, cv2.COLOR_GRAY2RGB)

        return Image.fromarray(sketch_rgb)

# ==========================================
# 2. Data Processing Pipeline and DataLoaders
# ==========================================

def get_cifar10_dataloaders(batch_size=64):
    """Get base and corrupted CIFAR-10 dataloaders"""

    base_transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
```

```
    ])

    # 1. Ideal Scenario
    train_dataset = torchvision.datasets.CIFAR10(root='./data', train=True,
download=True, transform=base_transform)
    test_clean = torchvision.datasets.CIFAR10(root='./data', train=False,
download=True, transform=base_transform)

    # 2. 50% Salt and Pepper Noise Scenario
    transform_noise = transforms.Compose([
        AddSaltPepperNoise(prob=0.5),
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
    ])
    test_noise = torchvision.datasets.CIFAR10(root='./data', train=False,
download=True, transform=transform_noise)

    # 3. 30% Spatial Occlusion Scenario
    transform_occlusion = transforms.Compose([
        transforms.ToTensor(),
        transforms.RandomErasing(p=1.0, scale=(0.3, 0.3), ratio=(1.0, 1.0),
value=0), # Strict 30% area
        transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
    ])
    test_occlusion = torchvision.datasets.CIFAR10(root='./data', train=False,
download=True, transform=transform_occlusion)

    # 4. Sketch Stylization Scenario
    transform_sketch = transforms.Compose([
        SketchStylization(),
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
    ])
    test_sketch = torchvision.datasets.CIFAR10(root='./data', train=False,
download=True, transform=transform_sketch)

    dataloaders = {
        'train': torch.utils.data.DataLoader(train_dataset, batch_size=batch_size,
shuffle=True, num_workers=0),
        'test_clean': torch.utils.data.DataLoader(test_clean,
batch_size=batch_size, shuffle=False, num_workers=0),
        'test_noise': torch.utils.data.DataLoader(test_noise,
batch_size=batch_size, shuffle=False, num_workers=0),
        'test_occlusion': torch.utils.data.DataLoader(test_occlusion,
batch_size=batch_size, shuffle=False, num_workers=0),
        'test_sketch': torch.utils.data.DataLoader(test_sketch,
batch_size=batch_size, shuffle=False, num_workers=0),
    }

    return dataloaders
```

```
# ==========================================
# 3. Core Visualization Module
# ==========================================
def visualize_transformations():
    print("⏳ Generating visualization comparison matrix for corrupted data...")

    # Get the raw dataset (no Normalize, keep displayable PIL format)
    dataset_raw = torchvision.datasets.CIFAR10(root='./data', train=False,
download=True)
    classes = dataset_raw.classes

    noise_transform = AddSaltPepperNoise(prob=0.5)
    sketch_transform = SketchStylization()
    transform_to_tensor = transforms.ToTensor()
    erasing_transform = transforms.RandomErasing(p=1.0, scale=(0.3, 0.3), ratio=
(1.0, 1.0), value=0)

    indices = [3, 4, 5, 14]

    fig, axes = plt.subplots(4, 4, figsize=(10, 10), dpi=200)
    fig.suptitle('CIFAR-10: Ideal vs Non-Ideal Scenarios', fontsize=18,
fontweight='bold', y=0.95)

    col_titles = ['Original (Clean)', 'Salt & Pepper Noise', 'Spatial Occlusion',
'Sketch Stylization']

    for i, idx in enumerate(indices):
        img_pil, label = dataset_raw[idx]

        # 1. Original Image
        img_clean = np.array(img_pil)

        # 2. Salt and Pepper Noise
        img_noise = np.array(noise_transform(img_pil))

        # 3. Occlusion (RandomErasing requires Tensor input)
        img_tensor = transform_to_tensor(img_pil)
        img_erased = erasing_transform(img_tensor)
        img_occlusion = img_erased.permute(1, 2, 0).numpy() # Convert back to
(H,W,C) for display

        # 4. Sketch Stylization
        img_sketch = np.array(sketch_transform(img_pil))

        images = [img_clean, img_noise, img_occlusion, img_sketch]

        for j in range(4):
            ax = axes[i, j]
            if j == 2:
```

```
                ax.imshow(images[j])
            else:
                ax.imshow(images[j].astype(np.uint8))

            ax.axis('off')

            if i == 0:
                ax.set_title(col_titles[j], fontsize=12, fontweight='bold', pad=10)

            if j == 0:
                ax.text(-0.2, 0.5, classes[label].capitalize(),
                        transform=ax.transAxes, fontsize=14, fontweight='bold',
                        va='center', ha='right', rotation=90)

    plt.tight_layout()
    plt.subplots_adjust(top=0.88, left=0.1)

    save_path = 'cifar10_corruptions_preview.png'
    plt.savefig(save_path, bbox_inches='tight', transparent=False)
    print(f"✅ Visualization matrix generated and safely saved to: {save_path}")
    plt.show()

# ==========================================
# 4. Main Execution Entry Point
# ==========================================
if __name__ == "__main__":
    visualize_transformations()

    print("\n" + "="*50)
    print("Preparing CIFAR-10 dataloaders with non-ideal scenario corruptions...")
    ldataloaders = get_cifar10_dataloaders(batch_size=128)
    print("Dataloaders ready! Includes the following test sets:")
    for key in loaders.keys():
        print(f" - {key.ljust(15)}: {len(loaders[key].dataset)} images")
```

Turn 2

User

```
import torch
import torch.nn as nn
from torchvision.models.vision_transformer import VisionTransformer
# ==========================================
# 0. 轻量级模型定义 (统一对齐至 ~1.7 M 级别)
# ==========================================
# 1. 传统 MLP
class MLPMini(nn.Module):
def __init__(self, input_dim=32*32*3, hidden_dim=512, num_classes=10):
super(MLPMini, self).__init__()
self.flatten = nn.Flatten()
self.network = nn.Sequential(
nn.Linear(input_dim, hidden_dim),
nn.ReLU(),
nn.Linear(hidden_dim, hidden_dim // 2),
nn.ReLU(),
nn.Linear(hidden_dim // 2, num_classes)
)
def forward(self, x):
x = self.flatten(x)
return self.network(x)
# 2. 微型 CNN
class CNNMini(nn.Module):
def __init__(self, num_classes=10):
super(CNNMini, self).__init__()
self.features = nn.Sequential(
nn.Conv2d(3, 32, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
nn.Conv2d(64, 128, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2)
```

```
)
self.classifier = nn.Sequential(
nn.Flatten(),
nn.Linear(128 * 4 * 4, 768),
nn.ReLU(),
nn.Linear(768, num_classes)
)
def forward(self, x):
x = self.features(x)
return self.classifier(x)
# 3. 微型 ViT
def get_micro_vit_model(num_classes=10):
model = VisionTransformer(
image_size=32,
patch_size=4,
num_layers=6,
num_heads=6,
hidden_dim=192,
mlp_dim=384,
num_classes=num_classes
)
return model
# ==========================================
# 测试与打印参数量
# ==========================================
def count_parameters(model):
return sum(p.numel() for p in model.parameters() if p.requires_grad)
if __name__ == "__main__":
print(">>> 正在初始化 3 种 1.7M 级别的轻量级深度学习架构...\n")
dummy_input = torch.randn(2, 3, 32, 32)
models_dict = {
```

"MLP_Mini": MLPMini(),

"CNN_Mini": CNNMini(),

"ViT_Micro": get_micro_vit_model()

}

print("=" * 65)

print("模型复杂度与前向传播测试 (同级别 1.7M 轻量对决)")

print("=" * 65)

print(f"{'架构名称':<15} | {'参数量 (Params)':<15} | {'输出维度':<12}")

print("-" * 65)

for name, model in models_dict.items():

output = model(dummy_input)

params_count = count_parameters(model) / 1e6

print(f"{name:<13} | {params_count:>8.2f} M | {str(list(output.shape)):<12}")

print("-" * 65)不修改代码，将注释和输出转换成英文

Assistant

Here is your code with all the comments and print statements translated to English, while keeping the logic and structure exactly the same:

```
 import torch
import torch.nn as nn
from torchvision.models.vision_transformer import VisionTransformer

# ==========================================
# 0. Lightweight Model Definitions (Uniformly aligned to ~1.7 M level)
# ==========================================

# 1. Traditional MLP
class MLPMini(nn.Module):
    def __init__(self, input_dim=32*32*3, hidden_dim=512, num_classes=10):
        super(MLPMini, self).__init__()
        self.flatten = nn.Flatten()
        self.network = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim // 2),
            nn.ReLU(),
            nn.Linear(hidden_dim // 2, num_classes)
        )

    def forward(self, x):
```

```
        x = self.flatten(x)
        return self.network(x)

# 2. Miniature CNN
class CNNMini(nn.Module):
    def __init__(self, num_classes=10):
        super(CNNMini, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.ReLU(),
nn.MaxPool2d(2),
            nn.Conv2d(64, 128, kernel_size=3, padding=1), nn.ReLU(),
nn.MaxPool2d(2)
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 4 * 4, 768),
            nn.ReLU(),
            nn.Linear(768, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        return self.classifier(x)

# 3. Miniature ViT
def get_micro_vit_model(num_classes=10):
    model = VisionTransformer(
        image_size=32,
        patch_size=4,
        num_layers=6,
        num_heads=6,
        hidden_dim=192,
        mlp_dim=384,
        num_classes=num_classes
    )
    return model

# ==========================================
# Test and print parameter counts
# ==========================================
def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

if __name__ == "__main__":
    print(">>> Initializing 3 lightweight deep learning architectures at the 1.7M
level...\n")

    dummy_input = torch.randn(2, 3, 32, 32)
```

```
    models_dict = {
        "MLP_Mini": MLPMini(),
        "CNN_Mini": CNNMini(),
        "ViT_Micro": get_micro_vit_model()
    }

    print("=" * 65)
    print("Model Complexity & Forward Pass Test (1.7M Lightweight Showdown)")
    print("=" * 65)
    print(f"{'Architecture':<15} | {'Params Count':<15} | {'Output Shape':<12}")
    print("-" * 65)

    for name, model in models_dict.items():
        output = model(dummy_input)
        params_count = count_parameters(model) / 1e6
        print(f"{name:<13} | {params_count:>8.2f} M        |
{str(list(output.shape)):<12}")

    print("-" * 65)
```

Turn 3

User

```
import torch
import torch.nn as nn
import torch.optim as optim
from pathlib import Path
from tqdm import tqdm
from torchvision.models.vision_transformer import VisionTransformer
# ==========================================
# 0. 轻量级模型定义 (统一对齐至 ~1.7 M 级别)
# ==========================================
class MLPMini(nn.Module):
def __init__(self, input_dim=32*32*3, hidden_dim=512, num_classes=10):
super(MLPMini, self).__init__()
self.flatten = nn.Flatten()
self.network = nn.Sequential(
nn.Linear(input_dim, hidden_dim),
nn.ReLU(),
nn.Linear(hidden_dim, hidden_dim // 2),
nn.ReLU(),
nn.Linear(hidden_dim // 2, num_classes)
)
def forward(self, x):
return self.network(self.flatten(x))
class CNNMini(nn.Module):
def __init__(self, num_classes=10):
super(CNNMini, self).__init__()
self.features = nn.Sequential(
nn.Conv2d(3, 32, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
nn.Conv2d(64, 128, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2)
```

```
)
self.classifier = nn.Sequential(
nn.Flatten(),
nn.Linear(128 * 4 * 4, 768),
nn.ReLU(),
nn.Linear(768, num_classes)
)
def forward(self, x):
return self.classifier(self.features(x))
def get_micro_vit_model(num_classes=10):
return VisionTransformer(
image_size=32, patch_size=4, num_layers=6, num_heads=6,
hidden_dim=192, mlp_dim=384, num_classes=num_classes
)
# ==========================================
# 1. 独立训练模块 (含自动加载已有权重机制)
# ==========================================
def run_training_phase(models_dict, dataloaders, num_epochs=30, device='cpu',
save_dir='./saved_models'):
print(f"\n{'='*60}\n🚀 [阶段一] 开始执行模型训练与封印\n{'='*60}")
base_path = Path(save_dir)
base_path.mkdir(parents=True, exist_ok=True)
for model_name, model in models_dict.items():
# 提前计算存放路径
safe_name = model_name.replace(" ", "_").lower()
model_save_path = base_path / f"{safe_name}_weights.pth"
# 🌟 核心判断逻辑：如果存在，直接加载；如果不存在，开始训练
if model_save_path.exists():
print(f"\n>>> ♻️ 发现本地已有权重文件，直接加载 [{model_name}]，跳过训练。")
model.load_state_dict(torch.load(str(model_save_path), map_location=device))
model = model.to(device)
continue # 跳过下面的训练循环，直接进入下一个模型
```

```
print(f"\n>>> 🔨 未检测到历史权重，正在从头训练: [{model_name}]")
model = model.to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
# 训练循环
for epoch in range(num_epochs):
model.train()
running_loss, correct, total = 0.0, 0, 0
train_loop = tqdm(dataloaders['train'], desc=f"Epoch [{epoch+1}/{num_epochs}]", colour='green')
for inputs, labels in train_loop:
inputs, labels = inputs.to(device), labels.to(device)
optimizer.zero_grad()
outputs = model(inputs)
loss = criterion(outputs, labels)
loss.backward()
optimizer.step()
running_loss += loss.item()
_, predicted = outputs.max(1)
total += labels.size(0)
correct += predicted.eq(labels).sum().item()
train_loop.set_postfix(loss=f"{loss.item():.4f}")
if device.type in ['mps', 'cuda']:
torch.cuda.empty_cache() if device.type == 'cuda' else torch.mps.empty_cache()
# 模型保存
torch.save(model.state_dict(), str(model_save_path))
print(f"💾 {model_name} 训练完成！权重已保存至: {model_save_path}")
# ==========================================
# 执行逻辑
# ==========================================
if __name__ == "__main__":
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "mps" if torch.backends.mps.is_available() else "cpu")
print(f"⚙️ 当前使用的计算设备: {device}")
models_to_train = {
"MLP_Mini": MLPMini(),
"CNN_Mini": CNNMini(),
"ViT_Micro": get_micro_vit_model()
}
try:
print("📦 正在准备数据加载器...")
# 注意：这里需要确保 get_cifar10_dataloaders 函数在你的代码上方已经定义
# 如果你是在 Jupyter 里，请确保你已经运行了包含该函数的那个 Cell
dataloaders = get_cifar10_dataloaders(batch_size=128)
print("✅ 数据准备完毕，准备进入训练阶段！")
run_training_phase(models_to_train, dataloaders, num_epochs=30, device=device)
except NameError as e:
print(f"⚠️ 致命错误: 找不到函数或变量。详细信息: {e}")
print("👉 提示：请确保包含了 get_cifar10_dataloaders 函数的代码已经被运行！")是否和上面代码冲突，是否一致
```

🤖 **Assistant**

这部分代码与你前面提供的两段代码**完全不冲突，并且在逻辑上高度一致**。它实际上是前两段代码的完美延续，构成了完整项目流程（数据预处理 -> 模型定义 -> 模型训练）的第三步。

以下是具体的对比和一致性分析：

1. 模型定义（完美对齐）

虽然你在这段代码中重新定义了 `MLPMini` 、 `CNNMini` 和 `get_micro_vit_model` ，但它们的网络层结构、参数维度和整体复杂度与上一段代码完全一致（依然保持在约 1.7M 参数量）。

- 细微区别：这段代码写得更加精简（例如将 `x = self.flatten(x)` 和 `return self.network(x)` 合并成了 `return self.network(self.flatten(x))` ），这是非常优雅的代码优化，没有任何逻辑冲突。

2. 接口调用（无缝衔接）

这段代码在主函数 `if __name__ == "__main__":` 中，通过以下这行代码完美调用了你在第一段代码中定义的函数：

```
dataloaders = get_cifar10_dataloaders(batch_size=128)
```

并且你非常有预见性地加入了 `try-except NameError` 捕获机制。这说明这段代码设计时就已经考虑到它是依赖第一段数据加载代码的。只要它们在同一个 Jupyter Notebook 的上下文中运行，或者被放在同一个 Python 文件中，就能完美闭环。

3. 新增亮点（平滑拓展）

这段代码基于前文，非常合理地引入了真正的训练逻辑，没有破坏前面的任何设定，并且增加了几个很好的工程化设计：

- **断点续训/自动加载**：通过检查 `.pth` 文件是否存在来跳过已完成的训练，这在对比实验中非常实用。
- **硬件自适应**：加入了对 `cuda` (NVIDIA), `mps` (Apple Silicon) 和 `cpu` 的自动检测，适应性强。
- **进度可视化**：引入了 `tqdm` ，让训练过程对用户可见。

总结建议

你可以放心地将这三段代码组合在一起运行。为了确保万无一失，如果你是在本地或者 Jupyter 环境中运行，建议的执行顺序是：

1. **运行第一段代码**：加载并测试数据增强与 `get_cifar10_dataloaders` 。
2. **运行第二段代码（可选）**：纯粹用于打印和检查模型参数量是否对齐。
3. **运行这第三段代码**：开始真正的训练与权重保存。

Turn 4

User

```
import torch
import torch.nn as nn
import torch.optim as optim
from pathlib import Path
from tqdm import tqdm
from torchvision.models.vision_transformer import VisionTransformer
class MLPMini(nn.Module):
def __init__(self, input_dim=32*32*3, hidden_dim=512, num_classes=10):
super(MLPMini, self).__init__()
self.flatten = nn.Flatten()
self.network = nn.Sequential(
nn.Linear(input_dim, hidden_dim),
nn.ReLU(),
nn.Linear(hidden_dim, hidden_dim // 2),
nn.ReLU(),
nn.Linear(hidden_dim // 2, num_classes)
)
def forward(self, x):
return self.network(self.flatten(x))
class CNNMini(nn.Module):
def __init__(self, num_classes=10):
super(CNNMini, self).__init__()
self.features = nn.Sequential(
nn.Conv2d(3, 32, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
nn.Conv2d(64, 128, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2)
)
self.classifier = nn.Sequential(
nn.Flatten(),
```

```
nn.Linear(128 * 4 * 4, 768),
nn.ReLU(),
nn.Linear(768, num_classes)
)
def forward(self, x):
return self.classifier(self.features(x))
def get_micro_vit_model(num_classes=10):
return VisionTransformer(
image_size=32, patch_size=4, num_layers=6, num_heads=6,
hidden_dim=192, mlp_dim=384, num_classes=num_classes
)
# ==========================================
# 1. 独立训练模块
# ==========================================
def run_training_phase(models_dict, dataloaders, num_epochs=30, device='cpu',
save_dir='./saved_models'):
print(f"\n{'='*60}\n🚀 [阶段一] 开始执行模型训练与封印\n{'='*60}")
base_path = Path(save_dir)
base_path.mkdir(parents=True, exist_ok=True)
for model_name, model in models_dict.items():
safe_name = model_name.replace(" ", "_").lower()
model_save_path = base_path / f"{safe_name}_weights.pth"
if model_save_path.exists():
print(f"\n>>> ♻️ 发现本地已有权重文件，直接加载 [{model_name}]，跳过训练。")
model.load_state_dict(torch.load(str(model_save_path), map_location=device))
model = model.to(device)
continue
print(f"\n>>> 🔨 未检测到历史权重，正在从头训练: [{model_name}]")
model = model.to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
for epoch in range(num_epochs):
```

```
model.train()
running_loss, correct, total = 0.0, 0, 0
train_loop = tqdm(dataloaders['train'], desc=f"Epoch [{epoch+1}/{num_epochs}]", colour='green')
for inputs, labels in train_loop:
inputs, labels = inputs.to(device), labels.to(device)
optimizer.zero_grad()
outputs = model(inputs)
loss = criterion(outputs, labels)
loss.backward()
optimizer.step()
running_loss += loss.item()
_, predicted = outputs.max(1)
total += labels.size(0)
correct += predicted.eq(labels).sum().item()
train_loop.set_postfix(loss=f"{loss.item():.4f}")
if device.type in ['mps', 'cuda']:
torch.cuda.empty_cache() if device.type == 'cuda' else torch.mps.empty_cache()
torch.save(model.state_dict(), str(model_save_path))
print(f"💾 {model_name} 训练完成！权重已保存至: {model_save_path}")
# ==========================================
# 执行逻辑
# ==========================================
if __name__ == "__main__":
device = torch.device("cuda" if torch.cuda.is_available() else "mps" if torch.backends.mps.is_available() else "cpu")
print(f"⚙️ 当前使用的计算设备: {device}")
models_to_train = {
"MLP_Mini": MLPMini(),
"CNN_Mini": CNNMini(),
"ViT_Micro": get_micro_vit_model()
}
```

> try:
>
> print("📦 正在准备数据加载器...")
>
> dataloaders = get_cifar10_dataloaders(batch_size=128)
>
> print("✅ 数据准备完毕，准备进入训练阶段！")
>
> run_training_phase(models_to_train, dataloaders, num_epochs=30, device=device)
>
> except NameError as e:
>
> print(f"⚠️ 致命错误: 找不到函数或变量。详细信息: {e}")
>
> print("👉 提示：请确保包含了 get_cifar10_dataloaders 函数的代码已经被运行！")删除所有emoji然后输出和注释为英文

🤖 Assistant

> Here is the code with all emojis removed, and the comments and output statements translated to English:
>
> ```
> import torch
> import torch.nn as nn
> import torch.optim as optim
> from pathlib import Path
> from tqdm import tqdm
> from torchvision.models.vision_transformer import VisionTransformer
>
> class MLPMini(nn.Module):
>  def __init__(self, input_dim=32*32*3, hidden_dim=512, num_classes=10):
>  super(MLPMini, self).__init__()
>  self.flatten = nn.Flatten()
>  self.network = nn.Sequential(
>  nn.Linear(input_dim, hidden_dim),
>  nn.ReLU(),
>  nn.Linear(hidden_dim, hidden_dim // 2),
>  nn.ReLU(),
>  nn.Linear(hidden_dim // 2, num_classes)
>  )
>  def forward(self, x):
>  return self.network(self.flatten(x))
>
> class CNNMini(nn.Module):
>  def __init__(self, num_classes=10):
>  super(CNNMini, self).__init__()
>  self.features = nn.Sequential(
>  nn.Conv2d(3, 32, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
>  nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.ReLU(),
> nn.MaxPool2d(2),
>  nn.Conv2d(64, 128, kernel_size=3, padding=1), nn.ReLU(),
> nn.MaxPool2d(2)
>  )
> ```

```
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 4 * 4, 768),
            nn.ReLU(),
            nn.Linear(768, num_classes)
        )
    def forward(self, x):
        return self.classifier(self.features(x))

def get_micro_vit_model(num_classes=10):
    return VisionTransformer(
        image_size=32, patch_size=4, num_layers=6, num_heads=6,
        hidden_dim=192, mlp_dim=384, num_classes=num_classes
    )

# ==========================================
# 1. Independent Training Module
# ==========================================
def run_training_phase(models_dict, dataloaders, num_epochs=30, device='cpu',
save_dir='./saved_models'):
    print(f"\n{'='*60}\n[Phase 1] Starting model training and sealing\n{'='*60}")
    base_path = Path(save_dir)
    base_path.mkdir(parents=True, exist_ok=True)

    for model_name, model in models_dict.items():
        safe_name = model_name.replace(" ", "_").lower()
        model_save_path = base_path / f"{safe_name}_weights.pth"

        if model_save_path.exists():
            print(f"\n>>> Found existing local weight file, loading [{model_name}]
directly and skipping training.")
            model.load_state_dict(torch.load(str(model_save_path),
map_location=device))
            model = model.to(device)
            continue

        print(f"\n>>> No historical weights detected, training from scratch:
[{model_name}]")
        model = model.to(device)
        criterion = nn.CrossEntropyLoss()
        optimizer = optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)

        for epoch in range(num_epochs):
            model.train()
            running_loss, correct, total = 0.0, 0, 0
            train_loop = tqdm(dataloaders['train'], desc=f"Epoch
[{epoch+1}/{num_epochs}]", colour='green')

            for inputs, labels in train_loop:
                inputs, labels = inputs.to(device), labels.to(device)
```

```
                optimizer.zero_grad()
                outputs = model(inputs)
                loss = criterion(outputs, labels)
                loss.backward()
                optimizer.step()

                running_loss += loss.item()
                _, predicted = outputs.max(1)
                total += labels.size(0)
                correct += predicted.eq(labels).sum().item()
                train_loop.set_postfix(loss=f"{loss.item():.4f}")

            if device.type in ['mps', 'cuda']:
                torch.cuda.empty_cache() if device.type == 'cuda' else
torch.mps.empty_cache()

        torch.save(model.state_dict(), str(model_save_path))
        print(f"{model_name} training complete! Weights saved to:
{model_save_path}")

# ==========================================
# Execution Logic
# ==========================================
if __name__ == "__main__":
    device = torch.device("cuda" if torch.cuda.is_available() else "mps" if
torch.backends.mps.is_available() else "cpu")
    print(f"Current compute device: {device}")

    models_to_train = {
         "MLP_Mini": MLPMini(),
         "CNN_Mini": CNNMini(),
         "ViT_Micro": get_micro_vit_model()
    }

    try:
        print("Preparing dataloaders...")
        dataloaders = get_cifar10_dataloaders(batch_size=128)
        print("Data preparation complete, ready to enter the training phase!")

        run_training_phase(models_to_train, dataloaders, num_epochs=30,
device=device)

    except NameError as e:
        print(f"Fatal error: Function or variable not found. Details: {e}")
        print("Hint: Please ensure the code containing the get_cifar10_dataloaders
function has been run!")
```

Turn 5

👤 User

```
import torch
import torch.nn.functional as F
import time
import json
from pathlib import Path
from sklearn.metrics import f1_score
import numpy as np
from tqdm import tqdm
def calculate_ece(probs, labels, n_bins=10):
confidences = np.max(probs, axis=1)
predictions = np.argmax(probs, axis=1)
accuracies = (predictions == labels)
ece = 0.0
bin_boundaries = np.linspace(0, 1, n_bins + 1)
for i in range(n_bins):
bin_lower = bin_boundaries[i]
bin_upper = bin_boundaries[i+1]
in_bin = (confidences > bin_lower) & (confidences <= bin_upper)
prop_in_bin = np.mean(in_bin)
if prop_in_bin > 0:
accuracy_in_bin = np.mean(accuracies[in_bin])
avg_confidence_in_bin = np.mean(confidences[in_bin])
ece += np.abs(avg_confidence_in_bin - accuracy_in_bin) * prop_in_bin
return ece * 100
def run_evaluation_phase(models_dict, dataloaders, device='cpu',
weights_dir='./saved_models', output_json='evaluation_results.json'):
print(f"\n{'='*65}")
print(f"🔬 [阶段二] 开始跨场景多维度评估 (含 ECE 校准误差)")
print(f"{'='*65}")
```

```
test_scenarios = ['test_clean', 'test_noise', 'test_occlusion', 'test_sketch']
all_results = {}
base_path = Path(weights_dir)
for model_name, model in models_dict.items():
all_results[model_name] = {}
safe_name = model_name.replace(" ", "_").lower()
weight_path = base_path / f"{safe_name}_weights.pth"
# 加载权重
if weight_path.exists():
model.load_state_dict(torch.load(weight_path, map_location=device))
model = model.to(device)
model.eval()
print(f"\n>>> 成功加载 {model_name}，开始评估...")
else:
print(f"\n⚠️ 找不到 {model_name} 的权重文件，跳过评估。")
continue
with torch.no_grad():
for scenario in test_scenarios:
all_preds, all_labels = [], []
all_probs = []
start_time = time.time()
eval_loop = tqdm(dataloaders[scenario], desc=f"Eval {scenario}", leave=False, colour='blue')
for inputs, labels in eval_loop:
inputs, labels = inputs.to(device), labels.to(device)
outputs = model(inputs)
probs = F.softmax(outputs, dim=1)
_, predicted = outputs.max(1)
all_preds.extend(predicted.cpu().numpy())
all_labels.extend(labels.cpu().numpy())
all_probs.extend(probs.cpu().numpy())
all_labels_np = np.array(all_labels)
```

all_preds_np = np.array(all_preds)

all_probs_np = np.array(all_probs)

inference_time = time.time() - start_time

acc = np.mean(all_preds_np == all_labels_np) * 100

f1 = f1_score(all_labels_np, all_preds_np, average='macro') * 100

ece = calculate_ece(all_probs_np, all_labels_np)

all_results[model_name][scenario] = {

'acc': round(acc, 2),

'f1': round(f1, 2),

'ece': round(ece, 2),

'time': round(inference_time, 2)

}

tqdm.write(f"📊 {scenario.ljust(15)} | Acc: {acc:5.2f}% | F1: {f1:5.2f}% | ECE: {ece:5.2f}% 📉 | Time: {inference_time:5.2f}s")

with open(output_json, 'w', encoding='utf-8') as f:

json.dump(all_results, f, indent=4)

print(f"\n✅ 所有评估完成！数据已导出至 {output_json}")

return all_results

if __name__ == "__main__":

try:

all_results = run_evaluation_phase(models_to_train, dataloaders, device=device)

except NameError:

print("⚠️ 请确保你的 models_to_train 和 dataloaders 已经定义并加载！")删除所有emoji，把注释和输出转化为英文，不修改代码

🤖 Assistant

Here is your code with all emojis removed, and the comments and output statements translated to English, while keeping the logic strictly unchanged:

```
import torch
import torch.nn.functional as F
import time
import json
from pathlib import Path
from sklearn.metrics import f1_score
import numpy as np
```

```
from tqdm import tqdm

def calculate_ece(probs, labels, n_bins=10):

    confidences = np.max(probs, axis=1)
    predictions = np.argmax(probs, axis=1)
    accuracies = (predictions == labels)

    ece = 0.0
    bin_boundaries = np.linspace(0, 1, n_bins + 1)

    for i in range(n_bins):
        bin_lower = bin_boundaries[i]
        bin_upper = bin_boundaries[i+1]

        in_bin = (confidences > bin_lower) & (confidences <= bin_upper)
        prop_in_bin = np.mean(in_bin)

        if prop_in_bin > 0:
            accuracy_in_bin = np.mean(accuracies[in_bin])
            avg_confidence_in_bin = np.mean(confidences[in_bin])
            ece += np.abs(avg_confidence_in_bin - accuracy_in_bin) * prop_in_bin

    return ece * 100

def run_evaluation_phase(models_dict, dataloaders, device='cpu',
weights_dir='./saved_models', output_json='evaluation_results.json'):
    print(f"\n{'='*65}")
    print(f"[Phase 2] Starting cross-scenario multi-dimensional evaluation
(including ECE calibration error)")
    print(f"{'='*65}")

    test_scenarios = ['test_clean', 'test_noise', 'test_occlusion', 'test_sketch']
    all_results = {}
    base_path = Path(weights_dir)

    for model_name, model in models_dict.items():
        all_results[model_name] = {}
        safe_name = model_name.replace(" ", "_").lower()
        weight_path = base_path / f"{safe_name}_weights.pth"

        # Load weights
        if weight_path.exists():
            model.load_state_dict(torch.load(weight_path, map_location=device))
            model = model.to(device)
            model.eval()
            print(f"\n>>> Successfully loaded {model_name}, starting
evaluation...")
        else:
            print(f"\nWeight file for {model_name} not found, skipping
```

```
evaluation.")
            continue

        with torch.no_grad():
            for scenario in test_scenarios:
                all_preds, all_labels = [], []
                all_probs = []
                start_time = time.time()

                eval_loop = tqdm(dataloaders[scenario], desc=f"Eval {scenario}",
leave=False, colour='blue')
                for inputs, labels in eval_loop:
                    inputs, labels = inputs.to(device), labels.to(device)

                    outputs = model(inputs)
                    probs = F.softmax(outputs, dim=1)
                    _, predicted = outputs.max(1)


                    all_preds.extend(predicted.cpu().numpy())
                    all_labels.extend(labels.cpu().numpy())
                    all_probs.extend(probs.cpu().numpy())

                all_labels_np = np.array(all_labels)
                all_preds_np = np.array(all_preds)
                all_probs_np = np.array(all_probs)


                inference_time = time.time() - start_time
                acc = np.mean(all_preds_np == all_labels_np) * 100
                f1 = f1_score(all_labels_np, all_preds_np, average='macro') * 100
                ece = calculate_ece(all_probs_np, all_labels_np)

                all_results[model_name][scenario] = {
                    'acc': round(acc, 2),
                    'f1': round(f1, 2),
                    'ece': round(ece, 2),
                    'time': round(inference_time, 2)
                }

                tqdm.write(f"  {scenario.ljust(15)} | Acc: {acc:5.2f}% | F1:
{f1:5.2f}% | ECE: {ece:5.2f}% | Time: {inference_time:5.2f}s")

    with open(output_json, 'w', encoding='utf-8') as f:
        json.dump(all_results, f, indent=4)
    print(f"\nAll evaluations complete! Data exported to {output_json}")
    return all_results


if __name__ == "__main__":
```

```
    try:
        all_results = run_evaluation_phase(models_to_train, dataloaders,
device=device)
    except NameError:
        print("Please ensure your models_to_train and dataloaders are defined and
loaded!")
```

Turn 6

👤 User

```
import json
import matplotlib.pyplot as plt
def run_visualization_phase(json_path='evaluation_results.json'):
print(f"\n{'='*60}\n📊 [阶段三] 读取 JSON，生成多维可视化报告\n{'='*60}")
try:
with open(json_path, 'r', encoding='utf-8') as f:
results = json.load(f)
except FileNotFoundError:
print(f"⚠️ 找不到 {json_path}，请先运行评估模块！")
return
scenarios = ['test_clean', 'test_noise', 'test_occlusion', 'test_sketch']
labels_map = {'test_clean': 'Clean\n(Ideal)', 'test_noise': 'Noise', 'test_occlusion': 'Occlusion', 'test_sketch': 'Sketch'}
model_names = list(results.keys())
colors = ['#95a5a6', '#3498db', '#e74c3c']
plt.style.use('seaborn-v0_8-whitegrid')
fig = plt.figure(figsize=(18, 6))
# ==========================================
# 图 1: Accuracy Bar Chart
# ==========================================
ax1 = plt.subplot(1, 3, 1)
x = np.arange(len(scenarios))
width = 0.25
for i, m in enumerate(model_names):
accs = [results[m][sc]['acc'] for sc in scenarios]
offset = (i - len(model_names)/2 + 0.5) * width
rects = ax1.bar(x + offset, accs, width, label=m, color=colors[i], edgecolor='black')
for rect in rects:
height = rect.get_height()
```

```
ax1.annotate(f'{height:.1f}', xy=(rect.get_x() + rect.get_width()/2, height),
xytext=(0, 2), textcoords="offset points", ha='center', va='bottom', fontsize=8)
ax1.set_title('Metric 1: Accuracy Degradation', fontweight='bold')
ax1.set_ylabel('Accuracy (%)')
ax1.set_xticks(x)
ax1.set_xticklabels([labels_map[sc] for sc in scenarios])
ax1.legend()
ax1.set_ylim(0, 100)
# ==========================================
# 图 2: F1-Score Radar Chart
# ==========================================
ax2 = plt.subplot(1, 3, 2, polar=True)
angles = np.linspace(0, 2 * np.pi, len(scenarios), endpoint=False).tolist()
angles += angles[:1]
for i, m in enumerate(model_names):
f1_scores = [results[m][sc]['f1'] for sc in scenarios]
f1_scores += f1_scores[:1]
ax2.plot(angles, f1_scores, color=colors[i], linewidth=2, label=m)
ax2.fill(angles, f1_scores, color=colors[i], alpha=0.15)
ax2.set_title('Metric 2: Macro F1-Score Footprint', fontweight='bold', pad=20)
ax2.set_xticks(angles[:-1])
ax2.set_xticklabels([labels_map[sc].replace('\n', ' ') for sc in scenarios])
ax2.set_ylim(0, 100)
# ==========================================
# 图 3: Efficiency Scatter
# ==========================================
ax3 = plt.subplot(1, 3, 3)
for i, m in enumerate(model_names):
clean_acc = results[m]['test_clean']['acc']
total_time = sum([results[m][sc]['time'] for sc in scenarios])
ax3.scatter(total_time, clean_acc, s=300, color=colors[i], edgecolor='black', label=m, zorder=5)
```

ax3.annotate(m, (total_time, clean_acc), xytext=(10, -5), textcoords='offset points', fontweight='bold')

ax3.set_title('Metric 3: Accuracy vs. Inference Time', fontweight='bold')

ax3.set_xlabel('Total Inference Time (Seconds) ↘ (Faster is better)')

ax3.set_ylabel('Clean Accuracy (%) ↖ (Higher is better)')

ax3.grid(True, linestyle='--', alpha=0.7)

plt.suptitle('Cross-Architecture Robustness & Efficiency Analysis', fontsize=16, fontweight='bold', y=1.05)

plt.tight_layout()

plt.savefig('comprehensive_robustness_analysis.png', dpi=300, bbox_inches='tight')

print("✅ 可视化完成！已生成包含三个维度的拼接大图 'comprehensive_robustness_analysis.png'")

plt.show()

if __name__ == "__main__":

run_visualization_phase('evaluation_results.json')删除emoji，将注释和输出换成英文

Assistant

Here is the code with all emojis removed and the comments/outputs translated to English, without altering the logic:

```
 import json
import matplotlib.pyplot as plt
import numpy as np

def run_visualization_phase(json_path='evaluation_results.json'):
    print(f"\n{'='*60}\n[Phase 3] Read JSON, generate multi-dimensional
visualization report\n{'='*60}")

    try:
        with open(json_path, 'r', encoding='utf-8') as f:
            results = json.load(f)
    except FileNotFoundError:
        print(f"Cannot find {json_path}, please run the evaluation module first!")
        return

    scenarios = ['test_clean', 'test_noise', 'test_occlusion', 'test_sketch']
    labels_map = {'test_clean': 'Clean\n(Ideal)', 'test_noise': 'Noise',
'test_occlusion': 'Occlusion', 'test_sketch': 'Sketch'}
    model_names = list(results.keys())
    colors = ['#95a5a6', '#3498db', '#e74c3c']

    plt.style.use('seaborn-v0_8-whitegrid')
```

```
    fig = plt.figure(figsize=(18, 6))

    # ==========================================
    # Figure 1: Accuracy Bar Chart
    # ==========================================
    ax1 = plt.subplot(1, 3, 1)
    x = np.arange(len(scenarios))
    width = 0.25

    for i, m in enumerate(model_names):
        accs = [results[m][sc]['acc'] for sc in scenarios]
        offset = (i - len(model_names)/2 + 0.5) * width
        rects = ax1.bar(x + offset, accs, width, label=m, color=colors[i],
edgecolor='black')

        for rect in rects:
            height = rect.get_height()
            ax1.annotate(f'{height:.1f}', xy=(rect.get_x() + rect.get_width()/2,
height),
                         xytext=(0, 2), textcoords="offset points", ha='center',
va='bottom', fontsize=8)

    ax1.set_title('Metric 1: Accuracy Degradation', fontweight='bold')
    ax1.set_ylabel('Accuracy (%)')
    ax1.set_xticks(x)
    ax1.set_xticklabels([labels_map[sc] for sc in scenarios])
    ax1.legend()
    ax1.set_ylim(0, 100)

    # ==========================================
    # Figure 2: F1-Score Radar Chart
    # ==========================================
    ax2 = plt.subplot(1, 3, 2, polar=True)
    angles = np.linspace(0, 2 * np.pi, len(scenarios), endpoint=False).tolist()
    angles += angles[:1]

    for i, m in enumerate(model_names):
        f1_scores = [results[m][sc]['f1'] for sc in scenarios]
        f1_scores += f1_scores[:1]
        ax2.plot(angles, f1_scores, color=colors[i], linewidth=2, label=m)
        ax2.fill(angles, f1_scores, color=colors[i], alpha=0.15)

    ax2.set_title('Metric 2: Macro F1-Score Footprint', fontweight='bold', pad=20)
    ax2.set_xticks(angles[:-1])
    ax2.set_xticklabels([labels_map[sc].replace('\n', ' ') for sc in scenarios])
    ax2.set_ylim(0, 100)

    # ==========================================
    # Figure 3: Efficiency Scatter
    # ==========================================
```

```
    ax3 = plt.subplot(1, 3, 3)
    for i, m in enumerate(model_names):
        clean_acc = results[m]['test_clean']['acc']

        total_time = sum([results[m][sc]['time'] for sc in scenarios])

        ax3.scatter(total_time, clean_acc, s=300, color=colors[i],
edgecolor='black', label=m, zorder=5)
        ax3.annotate(m, (total_time, clean_acc), xytext=(10, -5),
textcoords='offset points', fontweight='bold')

    ax3.set_title('Metric 3: Accuracy vs. Inference Time', fontweight='bold')
    ax3.set_xlabel('Total Inference Time (Seconds) ↘ (Faster is better)')
    ax3.set_ylabel('Clean Accuracy (%) ↖ (Higher is better)')
    ax3.grid(True, linestyle='--', alpha=0.7)

    plt.suptitle('Cross-Architecture Robustness & Efficiency Analysis',
fontsize=16, fontweight='bold', y=1.05)
    plt.tight_layout()
    plt.savefig('comprehensive_robustness_analysis.png', dpi=300,
bbox_inches='tight')
    print("Visualization complete! Generated a comprehensive composite image with
three dimensions: 'comprehensive_robustness_analysis.png'")
    plt.show()

if __name__ == "__main__":
    run_visualization_phase('evaluation_results.json')
```

(Note: I added import numpy as np at the top since np.arange and np.linspace are used in the code but numpy wasn't explicitly imported in your snippet).

Turn 7

User

```
import torch
import matplotlib.pyplot as plt
import numpy as np
from pathlib import Path
plt.rcParams['font.sans-serif'] = ['PingFang SC', 'Arial Unicode MS', 'Heiti TC']
plt.rcParams['axes.unicode_minus'] = False
def get_saliency_map(model, image_tensor, target_class):
model.eval()
image_tensor = image_tensor.clone().requires_grad_()
output = model(image_tensor)
score = output[0, target_class]
model.zero_grad()
score.backward()
saliency = image_tensor.grad.abs().squeeze().cpu()
saliency, _ = torch.max(saliency, dim=0)
saliency = (saliency - saliency.min()) / (saliency.max() - saliency.min() + 1e-8)
return saliency.numpy()
def visualize_all_scenarios_attention(models_dict, dataloaders, image_idx=0, device='cpu',
weights_dir='./saved_models'):
print(f"\n{'='*60}\n👁️ [阶段四] 生成 4x4 全场景 XAI 特征热力图矩阵\n{'='*60}")
scenarios = ['test_clean', 'test_noise', 'test_occlusion', 'test_sketch']
row_titles = ['Clean', 'Noisy (S&P)', 'Occlusion', 'Sketch']
model_names = list(models_dict.keys())
base_path = Path(weights_dir)
loaded_models = {}
for name, model in models_dict.items():
safe_name = name.replace(" ", "_").lower()
weight_path = base_path / f"{safe_name}_weights.pth"
if weight_path.exists():
```

```
model.load_state_dict(torch.load(weight_path, map_location=device))
loaded_models[name] = model.to(device)
fig, axes = plt.subplots(4, len(model_names) + 1, figsize=(16, 14))
plt.style.use('dark_background')
def plot_image(ax, img_tensor, title):
img_np = img_tensor.cpu().squeeze().permute(1, 2, 0).numpy()
mean, std = np.array([0.4914, 0.4822, 0.4465]), np.array([0.2023, 0.1994, 0.2010])
img_np = std * img_np + mean
img_np = np.clip(img_np, 0, 1)
ax.imshow(img_np)
ax.set_title(title, fontsize=14, fontweight='bold', color='white', pad=10)
ax.axis('off')
def plot_heatmap(ax, saliency, title):
ax.imshow(saliency, cmap='inferno')
ax.set_title(title, fontsize=14, color='white', pad=10)
ax.axis('off')
for row_idx, (scen, r_title) in enumerate(zip(scenarios, row_titles)):
img_tensor, target_class = dataloaders[scen].dataset[image_idx]
img_tensor = img_tensor.unsqueeze(0).to(device)
plot_image(axes[row_idx, 0], img_tensor, f"Input: {r_title}")
for col_idx, name in enumerate(model_names):
model = loaded_models[name]
saliency = get_saliency_map(model, img_tensor, target_class)
title = f"{name}" if row_idx == 0 else ""
plot_heatmap(axes[row_idx, col_idx + 1], saliency, title)
plt.suptitle("Comprehensive Saliency Map Comparison Across All Corruption Scenarios",
fontsize=20, fontweight='bold', color='white', y=0.98)
plt.tight_layout(rect=[0, 0, 1, 0.95])
save_path = 'xai_saliency_all_scenarios.png'
plt.savefig(save_path, dpi=300, bbox_inches='tight', facecolor=fig.get_facecolor())
print(f"\n✅ 4x4 完整版 XAI 图像已生成，保存为 '{save_path}'")
plt.show()
```

if __name__ == "__main__":

device = torch.device("cuda" if torch.cuda.is_available() else "mps" if torch.backends.mps.is_available() else "cpu")

models_to_test = {

"MLP_Mini": MLPMini(),

"CNN_Mini": CNNMini(),

"ViT_Micro": get_micro_vit_model()

}

try:

target_image_idx = 0

visualize_all_scenarios_attention(models_to_test, dataloaders, image_idx=target_image_idx, device=device)

except NameError:

print("⚠️ 请确保你在运行此代码前，已经加载了 dataloaders。")删除所有emoji，将输出和注释转换成英文

🤖 **Assistant**

Here is the code with all emojis removed and the outputs translated to English, keeping the original logic completely intact:

```
 import torch
import matplotlib.pyplot as plt
import numpy as np
from pathlib import Path

plt.rcParams['font.sans-serif'] = ['PingFang SC', 'Arial Unicode MS', 'Heiti TC']
plt.rcParams['axes.unicode_minus'] = False

def get_saliency_map(model, image_tensor, target_class):
    model.eval()
    image_tensor = image_tensor.clone().requires_grad_()

    output = model(image_tensor)
    score = output[0, target_class]

    model.zero_grad()
    score.backward()

    saliency = image_tensor.grad.abs().squeeze().cpu()
    saliency, _ = torch.max(saliency, dim=0)
    saliency = (saliency - saliency.min()) / (saliency.max() - saliency.min() + 1e-
8)
```

```
    return saliency.numpy()

def visualize_all_scenarios_attention(models_dict, dataloaders, image_idx=0,
device='cpu', weights_dir='./saved_models'):
    print(f"\n{'='*60}\n[Phase 4] Generate 4x4 multi-scenario XAI feature heatmap
matrix\n{'='*60}")

    scenarios = ['test_clean', 'test_noise', 'test_occlusion', 'test_sketch']
    row_titles = ['Clean', 'Noisy (S&P)', 'Occlusion', 'Sketch']
    model_names = list(models_dict.keys())
    base_path = Path(weights_dir)

    loaded_models = {}
    for name, model in models_dict.items():
        safe_name = name.replace(" ", "_").lower()
        weight_path = base_path / f"{safe_name}_weights.pth"
        if weight_path.exists():
            model.load_state_dict(torch.load(weight_path, map_location=device))
        loaded_models[name] = model.to(device)

    fig, axes = plt.subplots(4, len(model_names) + 1, figsize=(16, 14))
    plt.style.use('dark_background')

    def plot_image(ax, img_tensor, title):
        img_np = img_tensor.cpu().squeeze().permute(1, 2, 0).numpy()
        mean, std = np.array([0.4914, 0.4822, 0.4465]), np.array([0.2023, 0.1994,
0.2010])
        img_np = std * img_np + mean
        img_np = np.clip(img_np, 0, 1)
        ax.imshow(img_np)
        ax.set_title(title, fontsize=14, fontweight='bold', color='white', pad=10)
        ax.axis('off')

    def plot_heatmap(ax, saliency, title):
        ax.imshow(saliency, cmap='inferno')
        ax.set_title(title, fontsize=14, color='white', pad=10)
        ax.axis('off')

    for row_idx, (scen, r_title) in enumerate(zip(scenarios, row_titles)):
        img_tensor, target_class = dataloaders[scen].dataset[image_idx]
        img_tensor = img_tensor.unsqueeze(0).to(device)

        plot_image(axes[row_idx, 0], img_tensor, f"Input: {r_title}")

        for col_idx, name in enumerate(model_names):
            model = loaded_models[name]
            saliency = get_saliency_map(model, img_tensor, target_class)
            title = f"{name}" if row_idx == 0 else ""
            plot_heatmap(axes[row_idx, col_idx + 1], saliency, title)
```

```
    plt.suptitle("Comprehensive Saliency Map Comparison Across All Corruption
Scenarios", fontsize=20, fontweight='bold', color='white', y=0.98)
    plt.tight_layout(rect=[0, 0, 1, 0.95])

    save_path = 'xai_saliency_all_scenarios.png'
    plt.savefig(save_path, dpi=300, bbox_inches='tight',
facecolor=fig.get_facecolor())
    print(f"\n4x4 full XAI image generated, saved as '{save_path}'")
    plt.show()

if __name__ == "__main__":
    device = torch.device("cuda" if torch.cuda.is_available() else "mps" if
torch.backends.mps.is_available() else "cpu")

    models_to_test = {
         "MLP_Mini": MLPMini(),
         "CNN_Mini": CNNMini(),
         "ViT_Micro": get_micro_vit_model()
    }

    try:
        target_image_idx = 0

        visualize_all_scenarios_attention(models_to_test, dataloaders,
image_idx=target_image_idx, device=device)
    except NameError:
        print("Please ensure you have loaded dataloaders before running this
code.")
```

Turn 8

User

> ---NameError Traceback (most recent call last)Cell In[3], line 172 170 ldataloaders = get_cifar10_dataloaders(batch_size=128) 171 print("Dataloaders ready! Includes the following test sets:")--> 172 for key in loaders.keys(): 173 print(f" - {key.ljust(15)}: {len(loaders[key].dataset)} images")NameError: name 'loaders' is not defined

Assistant

This is a classic typo! In your code, there is a mismatch between the variable you defined and the variable you are trying to loop through.

On line 170, the variable is named **ldataloaders**, but on lines 172 and 173, you are calling **loaders** (which doesn't exist).

Here is the corrected final block of your code. You just need to change them all to `dataloaders` so they match perfectly:

```
# ==========================================
# 4. Main Execution Entry Point
# ==========================================
if __name__ == "__main__":
    visualize_transformations()

    print("\n" + "="*50)
    print("Preparing CIFAR-10 dataloaders with non-ideal scenario corruptions...")

    # FIX 1: Change 'ldataloaders' to 'dataloaders'
    dataloaders = get_cifar10_dataloaders(batch_size=128)

    print("Dataloaders ready! Includes the following test sets:")

    # FIX 2: Change 'loaders' to 'dataloaders'
    for key in dataloaders.keys():
        print(f" - {key.ljust(15)}: {len(dataloaders[key].dataset)} images")
```

Just replace those last few lines in your cell, run it again, and it will work perfectly!

Turn 9

User

插入分开的Step markdown

Assistant

这是一个基于你上传的 Jupyter Notebook（`时间序列分析_1_1.ipynb`）中涉及的库（`yfinance`，`pandas`，`sklearn`，`matplotlib` 等）整理的标准时间序列分析流程。你可以直接将以下分开的 Markdown 单元格复制并插入到你的 Notebook 中，让代码逻辑更加清晰：

Step 1: 环境配置与库导入

说明：在这个阶段，我们需要导入数据获取、数据处理、机器学习和可视化相关的核心依赖库，并完成全局参数设置（如解决 Matplotlib 中文字体显示问题）。

```
## 1. 环境配置与库导入 (Environment Setup & Imports)
导入 `yfinance` 获取金融数据，使用 `pandas` 进行时间序列对齐，以及加载 `sklearn` 用于后续的建模与评估。
```

Step 2: 数据获取与检视

说明：通过 yfinance 下载指定标的（如股票、指数）的历史价格数据，并检查数据的基本信息（缺失值、时间区间、数据格式）。

```
## 2. 数据获取与检视 (Data Acquisition & Inspection)
通过 API 获取原始时间序列数据，观察数据的时间跨度、特征维度以及是否存在缺失值 (NaN)。
```

Step 3: 探索性数据分析 (EDA) 与可视化

说明：绘制时间序列的走势图，观察数据的趋势性（Trend）、季节性（Seasonality）和周期性。可以使用移动平均线（MA）来平滑噪音。

```
## 3. 探索性数据分析 (EDA & Visualization)
绘制收盘价走势图、成交量变化图，初步判断序列的平稳性以及长期的演变趋势。
```

Step 4: 特征工程与数据预处理

说明：机器学习模型无法直接理解时间戳，我们需要提取时间特征（如星期几、月份），并构建滞后特征（Lag features）或滚动窗口特征（Rolling statistics）。此外，还需要对数据进行标准

化/归一化。

```
## 4. 特征工程与数据预处理 (Feature Engineering & Preprocessing)
- **构造滞后特征**: 使用过去 $N$ 天的数据作为特征。
- **目标变量定义**: 明确预测目标（例如下一天的收盘价或收益率）。
- **数据缩放**: 使用 `sklearn.preprocessing` 统一特征尺度。
```

Step 5: 训练集与测试集划分

说明：时间序列数据具有严格的先后顺序，绝对不能使用随机打乱（Shuffle） 的方式划分数据集，通常按时间截断（Time-based splitting）。

```
## 5. 数据集划分 (Train-Test Split)
按照时间顺序，将前 80% 的数据作为训练集，后 20% 的数据作为测试集，防止未来数据泄露（Data Leakage）。
```

Step 6: 机器学习模型构建与训练

说明：使用 sklearn 构建回归或分类模型（如线性回归、随机森林、SVR等）来拟合时间序列数据。

```
## 6. 模型构建与训练 (Model Building & Training)
实例化机器学习模型，并使用训练集数据拟合模型参数。
```

Step 7: 模型评估与结果可视化

说明：使用测试集进行预测，计算评估指标（如 RMSE, MAE, R^2），并将预测曲线与真实曲线绘制在同一张图上进行直观对比。

```
## 7. 模型评估与可视化对比 (Evaluation & Visual Comparison)
计算预测误差，绘制预测值与真实值的重叠走势图，评估模型在未知时间段的表现。
```

你是希望我帮你把这些 Markdown 文本直接嵌入到你具体的 Python 代码片段之间，还是需要我针对 Notebook 中的某个特定分析步骤提供代码示例？

Turn 10

👤 User

import math

try:

from efficient_kan import KAN

HAS_KAN = True

except ImportError:

HAS_KAN = False

print("⚠️ 警告: 未找到 efficient-kan 库。如果不安装，KAN 模型将被跳过。请运行: %pip install efficient-kan")删除emoji，转换为纯英文，不修改代码

🤖 Assistant

这是去除 Emoji 并将输出信息翻译为纯英文后的代码，代码的整体逻辑保持不变：

```
import math

try:
    from efficient_kan import KAN
    HAS_KAN = True
except ImportError:
    HAS_KAN = False
    print("Warning: efficient-kan library not found. If not installed, KAN models will be skipped. Please run: %pip install efficient-kan")
```

Turn 11

👤 User

```
# ==========================================
# 1：基础环境、类定义与数据准备
# ==========================================
import yfinance as yf
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from sklearn.preprocessing import StandardScaler
from torch.utils.data import DataLoader, TensorDataset
from tqdm.notebook import tqdm
import matplotlib.pyplot as plt
import math
# ------------------------------------------
# 1. 数据获取与特征工程
# ------------------------------------------
def get_optimized_timeseries_dataloaders(ticker="^GSPC", lookback=60, batch_size=32):
print(f"📈 正在从 Yahoo Finance 获取 [{ticker}] 的历史数据...")
data = yf.download(ticker, period="10y", progress=False)
df = pd.DataFrame()
df['Close'] = data['Close']
df['Open'] = data['Open']
df['High'] = data['High']
df['Low'] = data['Low']
df['Volume'] = data['Volume']
df['MA5'] = df['Close'].rolling(window=5).mean()
df['MA20'] = df['Close'].rolling(window=20).mean()
```

```
df['LogReturn'] = np.log(df['Close'] / df['Close'].shift(1))
df = df.dropna()
feature_cols = ['LogReturn', 'Open', 'High', 'Low', 'Volume', 'MA5', 'MA20']
features = df[feature_cols].values
target = df[['LogReturn']].values
split_idx = int(len(features) * 0.8)
feature_scaler = StandardScaler()
target_scaler = StandardScaler()
train_features = feature_scaler.fit_transform(features[:split_idx])
train_target = target_scaler.fit_transform(target[:split_idx])
test_features = feature_scaler.transform(features[split_idx:])
test_target = target_scaler.transform(target[split_idx:])
scaled_features = np.vstack((train_features, test_features))
scaled_target = np.vstack((train_target, test_target))
X, y = [], []
for i in range(len(scaled_features) - lookback):
X.append(scaled_features[i:(i + lookback), :])
y.append(scaled_target[i + lookback, 0])
X = torch.tensor(np.array(X), dtype=torch.float32)
y = torch.tensor(np.array(y), dtype=torch.float32).unsqueeze(1)
actual_split = split_idx - lookback
X_train, y_train = X[:actual_split], y[:actual_split]
X_test, y_test = X[actual_split:], y[actual_split:]
train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size=batch_size,
shuffle=True)
test_loader = DataLoader(TensorDataset(X_test, y_test), batch_size=batch_size, shuffle=False)
prev_close_prices = df['Close'].values[split_idx - 1 : -1]
actual_close_prices = df['Close'].values[split_idx : ]
print(f"✅ 数据准备完毕！训练集: {len(X_train)} 条，测试集: {len(X_test)} 条。窗口长度:
{lookback}")
return train_loader, test_loader, target_scaler, prev_close_prices, actual_close_prices,
len(feature_cols)
```

```
# ----------------------------------------
# 2. 模型类定义
# ----------------------------------------
SEQ_LEN = 60
FEATURE_DIM = 7
EMBED_DIM = 64
HIDDEN_DIM = 256
OUTPUT_DIM = 1
# ----------------------------------------
# TS_MLP
# ----------------------------------------
class TS_MLP(nn.Module):
def __init__(self, seq_len=SEQ_LEN, feature_dim=FEATURE_DIM):
super().__init__()
self.flatten = nn.Flatten()
# 420 -> 128 -> 256 -> 64 -> 1
self.fc1 = nn.Linear(seq_len * feature_dim, 128)
self.fc2 = nn.Linear(128, 256)
self.fc3 = nn.Linear(256, 64)
self.fc_out = nn.Linear(64, 1)
self.dropout = nn.Dropout(0.3)
def forward(self, x):
x = self.flatten(x)
x = F.relu(self.fc1(x))
x = self.dropout(F.relu(self.fc2(x)))
x = F.relu(self.fc3(x))
return self.fc_out(x)
# ----------------------------------------
# TS_CNN
# ----------------------------------------
class TS_CNN(nn.Module):
```

```
def __init__(self, seq_len=SEQ_LEN, feature_dim=FEATURE_DIM):
super().__init__()
self.conv1 = nn.Conv1d(feature_dim, 128, kernel_size=5, padding=2)
self.conv2 = nn.Conv1d(128, 128, kernel_size=3, padding=1)
self.conv3 = nn.Conv1d(128, 128, kernel_size=3, padding=1)
self.pool = nn.AdaptiveAvgPool1d(1)
self.dropout = nn.Dropout(0.3)
self.fc = nn.Linear(128, 1)
def forward(self, x):
x = x.transpose(1, 2)
x = F.relu(self.conv1(x))
x = F.relu(self.conv2(x))
x = F.relu(self.conv3(x))
x = self.pool(x).squeeze(-1)
x = self.dropout(x)
return self.fc(x)
# ----------------------------------------
# TS_Transformer
# ----------------------------------------
class PositionalEncoding(nn.Module):
def __init__(self, d_model, max_len=5000):
super().__init__()
pe = torch.zeros(max_len, d_model)
position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model))
pe[:, 0::2] = torch.sin(position * div_term)
pe[:, 1::2] = torch.cos(position * div_term)
self.register_buffer('pe', pe.unsqueeze(0))
def forward(self, x):
return x + self.pe[:, :x.size(1)]
class TS_Transformer(nn.Module):
```

```
def __init__(self, seq_len=SEQ_LEN, feature_dim=FEATURE_DIM, embed_dim=EMBED_DIM, num_heads=4):
super().__init__()
self.input_proj = nn.Linear(feature_dim, embed_dim)
self.pos_encoder = PositionalEncoding(embed_dim)
encoder_layer = nn.TransformerEncoderLayer(
d_model=embed_dim, nhead=num_heads, batch_first=True, dropout=0.2, dim_feedforward=HIDDEN_DIM
)
self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=2)
self.fc = nn.Linear(embed_dim, 1)
def forward(self, x):
x = self.input_proj(x)
x = self.pos_encoder(x)
x = self.transformer(x)
return self.fc(x.mean(dim=1))
# ----------------------------------------
# TS_Mamba
# ----------------------------------------
class SimpleMambaBlock(nn.Module):
def __init__(self, d_model=EMBED_DIM, d_state=16):
super().__init__()
self.in_proj = nn.Linear(d_model, d_model * 2)
self.conv1d = nn.Conv1d(d_model, d_model, kernel_size=4, padding=3, groups=d_model)
self.x_proj = nn.Linear(d_model, d_state * 2 + d_model)
self.dt_proj = nn.Linear(d_model, d_model)
self.out_proj = nn.Linear(d_model, d_model)
def forward(self, x):
B, L, D = x.shape
xz = self.in_proj(x)
x_in, z = xz.chunk(2, dim=-1)
x_conv = F.silu(self.conv1d(x_in.transpose(1, 2))[:, :, :L].transpose(1, 2))
```

```
x_proj = self.x_proj(x_conv)
delta, A, B_val = torch.split(x_proj, [D, 16, 16], dim=-1)
return self.out_proj((x_conv * F.softplus(self.dt_proj(delta))) * F.silu(z))
class TS_Mamba(nn.Module):
def __init__(self, seq_len=SEQ_LEN, feature_dim=FEATURE_DIM,
embed_dim=EMBED_DIM, num_layers=4):
super().__init__()
self.proj = nn.Linear(feature_dim, embed_dim)
self.layers = nn.ModuleList([SimpleMambaBlock(embed_dim) for _ in range(num_layers)])
self.norms = nn.ModuleList([nn.LayerNorm(embed_dim) for _ in range(num_layers)])
self.fc = nn.Linear(embed_dim, 1)
def forward(self, x):
x = self.proj(x)
for layer, norm in zip(self.layers, self.norms):
x = norm(x + layer(x))
return self.fc(x.mean(dim=1))
# ----------------------------------------
# TS_KAN
# ----------------------------------------
class TS_KAN(nn.Module):
def __init__(self, seq_len=SEQ_LEN, feature_dim=FEATURE_DIM):
super().__init__()
self.flatten = nn.Flatten()
if HAS_KAN:
self.kan = KAN(
[seq_len * feature_dim, 26, 16, 1],
grid_size=5,
spline_order=3
)
else:
self.kan = None
def forward(self, x):
```

```
x = self.flatten(x)
return self.kan(x) if self.kan else torch.zeros(x.shape[0], 1)
# ----------------------------------------
# 3. 统一训练与评估函数
# ----------------------------------------
def evaluate_model(model, test_loader, target_scaler, device):
model.eval()
predictions, actuals = [], []
with torch.no_grad():
for X_batch, y_batch in test_loader:
predictions.extend(model(X_batch.to(device)).cpu().numpy())
actuals.extend(y_batch.numpy())
predictions = target_scaler.inverse_transform(predictions).flatten()
actuals = target_scaler.inverse_transform(actuals).flatten()
return predictions, actuals, np.mean((predictions - actuals)**2),
np.mean(np.sign(predictions)==np.sign(actuals))*100
def train_and_eval(model_name, model, loaders, target_scaler, device, epochs=30):
print(f"\n{'='*50}\n🚀 开始训练: [{model_name}]")
model.to(device)
optimizer = optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
model.train()
epoch_loop = tqdm(range(epochs), desc=f"训练进度", colour='green')
for epoch in epoch_loop:
train_loss = 0
for X_batch, y_batch in loaders['train']:
X_batch, y_batch = X_batch.to(device), y_batch.to(device)
optimizer.zero_grad()
loss = nn.MSELoss()(model(X_batch), y_batch)
loss.backward()
optimizer.step()
train_loss += loss.item()
epoch_loop.set_postfix(loss=f"{train_loss/len(loaders['train']):.6f}")
```

preds, _, mse, da = evaluate_model(model, loaders['test'], target_scaler, device)

print(f"📊 [{model_name}] 训练完成！测试集 MSE: {mse:.6f} | 方向准确率 (DA): {da:.2f}%")

return preds, mse, da

----

4. 初始化数据

----

device = torch.device("cuda" if torch.cuda.is_available() else "mps" if torch.backends.mps.is_available() else "cpu")

print(f"⚙️ 当前计算设备: {device}")

lookback_window = 60

train_loader, test_loader, target_scaler, prev_close_prices, actual_close_prices, feature_dim = get_optimized_timeseries_dataloaders(lookback=lookback_window)

loaders = {'train': train_loader, 'test': test_loader}

all_raw_preds = {}

all_da = {}

print("\n✅ 第一阶段：环境与数据准备已就绪，请运行下一代码块。")删除所有emoji，所有注释和输出转化成英文，不修改任何代码

🤖 Assistant

```
# ==========================================
# 1: Basic Environment, Class Definitions, and Data Preparation
# ==========================================
import yfinance as yf
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from sklearn.preprocessing import StandardScaler
from torch.utils.data import DataLoader, TensorDataset
from tqdm.notebook import tqdm
import matplotlib.pyplot as plt
import math

# ----------
# 1. Data Acquisition and Feature Engineering
# ----------
def get_optimized_timeseries_dataloaders(ticker="^GSPC", lookback=60,
batch_size=32):
```

```
    print(f"Fetching historical data for [{ticker}] from Yahoo Finance...")
    data = yf.download(ticker, period="10y", progress=False)

    df = pd.DataFrame()
    df['Close'] = data['Close']
    df['Open'] = data['Open']
    df['High'] = data['High']
    df['Low'] = data['Low']
    df['Volume'] = data['Volume']

    df['MA5'] = df['Close'].rolling(window=5).mean()
    df['MA20'] = df['Close'].rolling(window=20).mean()
    df['LogReturn'] = np.log(df['Close'] / df['Close'].shift(1))
    df = df.dropna()

    feature_cols = ['LogReturn', 'Open', 'High', 'Low', 'Volume', 'MA5', 'MA20']
    features = df[feature_cols].values
    target = df[['LogReturn']].values

    split_idx = int(len(features) * 0.8)
    feature_scaler = StandardScaler()
    target_scaler = StandardScaler()

    train_features = feature_scaler.fit_transform(features[:split_idx])
    train_target = target_scaler.fit_transform(target[:split_idx])
    test_features = feature_scaler.transform(features[split_idx:])
    test_target = target_scaler.transform(target[split_idx:])

    scaled_features = np.vstack((train_features, test_features))
    scaled_target = np.vstack((train_target, test_target))

    X, y = [], []
    for i in range(len(scaled_features) - lookback):
        X.append(scaled_features[i:(i + lookback), :])
        y.append(scaled_target[i + lookback, 0])

    X = torch.tensor(np.array(X), dtype=torch.float32)
    y = torch.tensor(np.array(y), dtype=torch.float32).unsqueeze(1)

    actual_split = split_idx - lookback
    X_train, y_train = X[:actual_split], y[:actual_split]
    X_test, y_test = X[actual_split:], y[actual_split:]

    train_loader = DataLoader(TensorDataset(X_train, y_train),
batch_size=batch_size, shuffle=True)
    test_loader = DataLoader(TensorDataset(X_test, y_test), batch_size=batch_size,
shuffle=False)

    prev_close_prices = df['Close'].values[split_idx - 1 : -1]
    actual_close_prices = df['Close'].values[split_idx : ]
```

```
    print(f"Data preparation complete! Training set: {len(X_train)} samples, Test
set: {len(X_test)} samples. Window length: {lookback}")
    return train_loader, test_loader, target_scaler, prev_close_prices,
actual_close_prices, len(feature_cols)

# ------------------------------------------
# 2. Model Class Definitions
# ------------------------------------------
SEQ_LEN = 60
FEATURE_DIM = 7
EMBED_DIM = 64
HIDDEN_DIM = 256
OUTPUT_DIM = 1

# ------------------------------------------
# TS_MLP
# ------------------------------------------
class TS_MLP(nn.Module):
    def __init__(self, seq_len=SEQ_LEN, feature_dim=FEATURE_DIM):
        super().__init__()
        self.flatten = nn.Flatten()
        # 420 -> 128 -> 256 -> 64 -> 1
        self.fc1 = nn.Linear(seq_len * feature_dim, 128)
        self.fc2 = nn.Linear(128, 256)
        self.fc3 = nn.Linear(256, 64)
        self.fc_out = nn.Linear(64, 1)
        self.dropout = nn.Dropout(0.3)

    def forward(self, x):
        x = self.flatten(x)
        x = F.relu(self.fc1(x))
        x = self.dropout(F.relu(self.fc2(x)))
        x = F.relu(self.fc3(x))
        return self.fc_out(x)

# ------------------------------------------
# TS_CNN
# ------------------------------------------
class TS_CNN(nn.Module):
    def __init__(self, seq_len=SEQ_LEN, feature_dim=FEATURE_DIM):
        super().__init__()
        self.conv1 = nn.Conv1d(feature_dim, 128, kernel_size=5, padding=2)
        self.conv2 = nn.Conv1d(128, 128, kernel_size=3, padding=1)
        self.conv3 = nn.Conv1d(128, 128, kernel_size=3, padding=1)
        self.pool = nn.AdaptiveAvgPool1d(1)
        self.dropout = nn.Dropout(0.3)
        self.fc = nn.Linear(128, 1)

    def forward(self, x):
```

```
        x = x.transpose(1, 2)
        x = F.relu(self.conv1(x))
        x = F.relu(self.conv2(x))
        x = F.relu(self.conv3(x))
        x = self.pool(x).squeeze(-1)
        x = self.dropout(x)
        return self.fc(x)

# ------------------------------------------
# TS_Transformer
# ------------------------------------------
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=5000):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-
math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        self.register_buffer('pe', pe.unsqueeze(0))

    def forward(self, x):
        return x + self.pe[:, :x.size(1)]

class TS_Transformer(nn.Module):
    def __init__(self, seq_len=SEQ_LEN, feature_dim=FEATURE_DIM,
embed_dim=EMBED_DIM, num_heads=4):
        super().__init__()
        self.input_proj = nn.Linear(feature_dim, embed_dim)
        self.pos_encoder = PositionalEncoding(embed_dim)
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=embed_dim, nhead=num_heads, batch_first=True, dropout=0.2,
dim_feedforward=HIDDEN_DIM
        )
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=2)
        self.fc = nn.Linear(embed_dim, 1)

    def forward(self, x):
        x = self.input_proj(x)
        x = self.pos_encoder(x)
        x = self.transformer(x)
        return self.fc(x.mean(dim=1))

# ------------------------------------------
# TS_Mamba
# ------------------------------------------
class SimpleMambaBlock(nn.Module):
    def __init__(self, d_model=EMBED_DIM, d_state=16):
        super().__init__()
```

```
        self.in_proj = nn.Linear(d_model, d_model * 2)
        self.conv1d = nn.Conv1d(d_model, d_model, kernel_size=4, padding=3,
groups=d_model)
        self.x_proj = nn.Linear(d_model, d_state * 2 + d_model)
        self.dt_proj = nn.Linear(d_model, d_model)
        self.out_proj = nn.Linear(d_model, d_model)

    def forward(self, x):
        B, L, D = x.shape
        xz = self.in_proj(x)
        x_in, z = xz.chunk(2, dim=-1)
        x_conv = F.silu(self.conv1d(x_in.transpose(1, 2))[:, :, :L].transpose(1,
2))
        x_proj = self.x_proj(x_conv)
        delta, A, B_val = torch.split(x_proj, [D, 16, 16], dim=-1)
        return self.out_proj((x_conv * F.softplus(self.dt_proj(delta))) *
F.silu(z))

class TS_Mamba(nn.Module):
    def __init__(self, seq_len=SEQ_LEN, feature_dim=FEATURE_DIM,
embed_dim=EMBED_DIM, num_layers=4):
        super().__init__()
        self.proj = nn.Linear(feature_dim, embed_dim)
        self.layers = nn.ModuleList([SimpleMambaBlock(embed_dim) for _ in
range(num_layers)])
        self.norms = nn.ModuleList([nn.LayerNorm(embed_dim) for _ in
range(num_layers)])
        self.fc = nn.Linear(embed_dim, 1)

    def forward(self, x):
        x = self.proj(x)
        for layer, norm in zip(self.layers, self.norms):
            x = norm(x + layer(x))
        return self.fc(x.mean(dim=1))

# ------------------------------------------
# TS_KAN
# ------------------------------------------
class TS_KAN(nn.Module):
    def __init__(self, seq_len=SEQ_LEN, feature_dim=FEATURE_DIM):
        super().__init__()
        self.flatten = nn.Flatten()
        if HAS_KAN:

            self.kan = KAN(
                [seq_len * feature_dim, 26, 16, 1],
                grid_size=5,
                spline_order=3
            )
        else:
```

```
            self.kan = None

    def forward(self, x):
        x = self.flatten(x)
        return self.kan(x) if self.kan else torch.zeros(x.shape[0], 1)

# ------------------------------------------
# 3. Unified Training and Evaluation Functions
# ------------------------------------------
def evaluate_model(model, test_loader, target_scaler, device):
    model.eval()
    predictions, actuals = [], []
    with torch.no_grad():
        for X_batch, y_batch in test_loader:
            predictions.extend(model(X_batch.to(device)).cpu().numpy())
            actuals.extend(y_batch.numpy())

    predictions = target_scaler.inverse_transform(predictions).flatten()
    actuals = target_scaler.inverse_transform(actuals).flatten()
    return predictions, actuals, np.mean((predictions - actuals)**2),
np.mean(np.sign(predictions)==np.sign(actuals))*100

def train_and_eval(model_name, model, loaders, target_scaler, device, epochs=30):
    print(f"\n{'='*50}\nStarting training: [{model_name}]")
    model.to(device)
    optimizer = optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
    model.train()

    epoch_loop = tqdm(range(epochs), desc=f"Training Progress", colour='green')
    for epoch in epoch_loop:
        train_loss = 0
        for X_batch, y_batch in loaders['train']:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            optimizer.zero_grad()
            loss = nn.MSELoss()(model(X_batch), y_batch)
            loss.backward()
            optimizer.step()
            train_loss += loss.item()
        epoch_loop.set_postfix(loss=f"{train_loss/len(loaders['train']):.6f}")

    preds, _, mse, da = evaluate_model(model, loaders['test'], target_scaler,
device)
    print(f"[{model_name}] Training complete! Test set MSE: {mse:.6f} | Directional
Accuracy (DA): {da:.2f}%")
    return preds, mse, da

# ------------------------------------------
# 4. Initialize Data
# ------------------------------------------
device = torch.device("cuda" if torch.cuda.is_available() else "mps" if
```

```
torch.backends.mps.is_available() else "cpu")
print(f"Current compute device: {device}")

lookback_window = 60
train_loader, test_loader, target_scaler, prev_close_prices, actual_close_prices,
feature_dim = get_optimized_timeseries_dataloaders(lookback=lookback_window)
loaders = {'train': train_loader, 'test': test_loader}

all_raw_preds = {}
all_da = {}
print("\nPhase 1: Environment and data preparation is ready, please run the next
code block.")
```

Turn 12

👤 User

> def count_parameters(model):
>
> return sum(p.numel() for p in model.parameters() if p.requires_grad)
>
> def format_params(num_params):
>
> return f"{num_params / 1e6:.4f} M"
>
> print("正在初始化模型以计算参数量...")
>
> models_to_test = {
>
> "TS_MLP": TS_MLP(),
>
> "TS_CNN": TS_CNN(),
>
> "TS_Transformer": TS_Transformer(),
>
> "TS_Mamba": TS_Mamba(),
>
> }
>
> if HAS_KAN:
>
> models_to_test["TS_KAN"] = TS_KAN()
>
> print(f"\n{'='*55}")
>
> print(f"🧠 模型参数量精确统计 (当前超参数下)")
>
> print(f"{'='*55}")
>
> print(f"{'模型名称 (Model)':<18} | {'统一单位 (M)':<12} | {'精确数量 (Exact)'}")
>
> print(f"{'-'*55}")
>
> for name, model in models_to_test.items():
>
> params = count_parameters(model)
>
> print(f"🔷 {name:<16} | {format_params(params):<12} | {params:,}")
>
> print(f"{'='*55}")

🤖 Assistant

> 这是去除所有 emoji 并将输出和注释翻译为英文后的代码，代码逻辑未做任何修改：
>
> ```
> def count_parameters(model):
>  return sum(p.numel() for p in model.parameters() if p.requires_grad)
>
> def format_params(num_params):
>  return f"{num_params / 1e6:.4f} M"
>
> print("Initializing models to calculate parameter counts...")
> ```

```
models_to_test = {
    "TS_MLP": TS_MLP(),
    "TS_CNN": TS_CNN(),
    "TS_Transformer": TS_Transformer(),
    "TS_Mamba": TS_Mamba(),
}

if HAS_KAN:
    models_to_test["TS_KAN"] = TS_KAN()

print(f"\n{'='*55}")
print(f"Exact Model Parameter Count Statistics (Under current hyperparameters)")
print(f"{'='*55}")
print(f"{'Model Name':<18} | {'Unit (M)':<12} | {'Exact Count'}")
print(f"{'-'*55}")

for name, model in models_to_test.items():
    params = count_parameters(model)
    print(f"  {name:<16} | {format_params(params):<12} | {params:,}")

print(f"{'='*55}")
```

Turn 13

👤 User

==========

前四个模型的训练

==========

print(">>> 开始执行四大经典模型训练管道...")

models_to_run = {

"TS_MLP": TS_MLP(seq_len=lookback_window, feature_dim=feature_dim),

"TS_CNN": TS_CNN(seq_len=lookback_window, feature_dim=feature_dim),

"TS_Transformer": TS_Transformer(seq_len=lookback_window, feature_dim=feature_dim),

"TS_Mamba": TS_Mamba(seq_len=lookback_window, feature_dim=feature_dim)

}

for name, model in models_to_run.items():

preds_log_returns, mse, da = train_and_eval(name, model, loaders, target_scaler, device, epochs=30)

all_raw_preds[name] = preds_log_returns

all_da[name] = da

print("\n✅ 第二阶段：四大经典模型训练完成，请运行下一代码块。")

🤖 Assistant

这是去除所有 emoji 并将注释和输出提示翻译为英文后的代码，核心逻辑完全保持不变：

```
 # ==========
# Training of the first four models
# ==========
print(">>> Starting the training pipeline for the four classic models...")

models_to_run = {
    "TS_MLP": TS_MLP(seq_len=lookback_window, feature_dim=feature_dim),
    "TS_CNN": TS_CNN(seq_len=lookback_window, feature_dim=feature_dim),
    "TS_Transformer": TS_Transformer(seq_len=lookback_window,
feature_dim=feature_dim),
    "TS_Mamba": TS_Mamba(seq_len=lookback_window, feature_dim=feature_dim)
}

for name, model in models_to_run.items():
    preds_log_returns, mse, da = train_and_eval(name, model, loaders,
target_scaler, device, epochs=30)
    all_raw_preds[name] = preds_log_returns
```

```
    all_da[name] = da

print("\nPhase 2: Training of the four classic models complete, please run the next code block.")
```

Turn 14

User

==========

KAN 模型训练

==========

if HAS_KAN:

print(">>> 开始执行 KAN 模型训练...")

kan_model = TS_KAN(seq_len=lookback_window, feature_dim=feature_dim)

preds_log_returns, mse, da = train_and_eval("TS_KAN", kan_model, loaders, target_scaler, device, epochs=30)

all_raw_preds["TS_KAN"] = preds_log_returns

all_da["TS_KAN"] = da

else:

print("⚠️ HAS_KAN 为 False，跳过 KAN 模型，直接绘制其他模型图表...")

Assistant

这是去除所有 emoji 并将注释和输出提示翻译为英文后的代码，逻辑保持不变：

```
 # ==========
# KAN Model Training
# ==========
if HAS_KAN:
    print(">>> Starting KAN model training...")
    kan_model = TS_KAN(seq_len=lookback_window, feature_dim=feature_dim)
    preds_log_returns, mse, da = train_and_eval("TS_KAN", kan_model, loaders, target_scaler, device, epochs=30)
    all_raw_preds["TS_KAN"] = preds_log_returns
    all_da["TS_KAN"] = da
else:
    print("HAS_KAN is False, skipping KAN model and directly plotting charts for other models...")
```

Turn 15

User

插入markdown step

Assistant

这是一个为您整理的 Jupyter Notebook Markdown 步骤说明。您可以将这些 Markdown 单元格复制并插入到您代码中对应的上方，以使整个实验的逻辑层次更加清晰、专业。

插入到第 1 个代码块（`!pip install ...`）上方

```
## Step 1: 环境配置与依赖安装
本节负责安装实验所需的深度学习框架（PyTorch）、模型库（Transformers, torchvision）、数据集工具（Datasets）以及状态空间模型 Mamba 的专属依赖（mamba-ssm, causal-conv1d）。
```

插入到第 2 个代码块（`import torch... is_available`）上方

```
## Step 2: 硬件加速检测
检测当前环境是否支持 GPU 加速（NVIDIA CUDA 或 Apple Metal MPS）。在处理极长文本特征提取时，GPU 将显著缩短运行时间。
```

插入到第 3 个代码块（包含全局配置、加载 PG19、文本转图像工具）上方

```
## Step 3: 全局参数配置与数据处理模块
本节定义了实验的核心参数和工具函数：
1. **全局配置**：设定上下文长度梯度（1024 到 16384）和目标记忆密钥。
2. **数据集加载**：流式加载 PG19 书籍级长文本，提取超长自然语言序列。
3. **“大海捞针”构造器**：在指定长度的上下文中随机截断并插入唯一密钥。
4. **Text2Image 转换器**：为了统一无归纳偏置（MLP）和视觉归纳偏置（ResNet/ViT）的输入维度，将文本的 ASCII 编码映射为 224x224 的 RGB 图像。
```

插入到第 4 个代码块（`PureMambaMemoryTester`）上方

```
## Step 4: 模型 1 - Mamba（状态空间模型）测试器
基于纯 PyTorch 实现的 Mamba 模块（替代需特定环境编译的 CUDA 算子）。该模型接收原生文本输入，通过计算长文本最后一层隐藏态特征与密钥隐藏态特征的余弦相似度，测试其原生时序建模的特征匹配能力。
```

插入到第 5 个代码块（`ViTMemoryTester`）上方

```
## Step 5: 模型 2 - ViT（视觉 Transformer）测试器
加载预训练的 `vit-base-patch16-224`。ViT 具备自注意力机制（Self-Attention），提取输入文本图像
```

> 的全局 `[CLS]` Token 作为特征进行相似度比对，测试其在长上下文下的注意力聚焦能力。

插入到第 6 个代码块（ ResNetMemoryTester ）上方

> ## Step 6: 模型 3 - ResNet50 (卷积神经网络) 测试器
> 加载预训练的 `ResNet50` 并移除最后的分类层以提取 2048 维特征。通过局部卷积操作处理文本图像，测试强局部归纳偏置架构在极长跨度记忆任务中的表现及衰减规律。

插入到第 7 个代码块（ MLPMemoryTester ）上方

> ## Step 7: 模型 4 - MLP (多层感知机) 测试器
> 构建一个约 10M 参数的轻量级纯全连接网络。通过 `Flatten` 直接展平文本图像的所有像素。作为无任何局部或时序归纳偏置的基线模型，测试其在纯特征映射下的记忆稳定性。

插入到第 8 个代码块（ run_experiment() 实验主流程）上方

> ## Step 8: 实验主流程与可视化评估
> 执行"大海捞针"核心逻辑:
> 1. 初始化四大模型测试器。
> 2. 遍历设定的上下文长度梯度 (1024, 4096, 8192, 16384)。
> 3. 每个长度下进行 20 次独立随机实验，记录余弦相似度命中率。
> 4. 绘制并保存《特征相似度 vs 上下文长度》对比曲线，输出格式化的实验数据报告。

Turn 16

👤 User

import torch

is_available = torch.cuda.is_available()

print("是否可用GPU:", is_available)

if is_available:

print("GPU型号:", torch.cuda.get_device_name(0))删除emoji，转换成英文，不修改代码

🤖 Assistant

Here is the code with the print statements translated to English. The logic remains completely unchanged:

```
import torch

is_available = torch.cuda.is_available()
print("Is GPU available:", is_available)

if is_available:
    print("GPU Model:", torch.cuda.get_device_name(0))
```

Turn 17

User

```
import torch
import torch.nn as nn
import numpy as np
import matplotlib.pyplot as plt
from tqdm import tqdm
from datasets import load_dataset
from PIL import Image
from transformers import AutoTokenizer, ViTImageProcessor, ViTForImageClassification
from torchvision import models, transforms
# ====================== 1. 全局配置 ======================
if torch.cuda.is_available():
DEVICE = "cuda"
elif torch.backends.mps.is_available():
DEVICE = "mps"
else:
DEVICE = "cpu"
print(f"当前使用的计算设备: {DEVICE}")
CONTEXT_LENGTHS = [1024, 4096, 8192, 16384]
NEEDLE_KEY = "实验密钥：PG19_MEMORY_TEST_2026"
PG19_SPLIT = "test"
PLOT_SAVE_PATH = "memory_accuracy_curve.png"
plt.rcParams['font.sans-serif'] = ['DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False
# ====================== 2. 加载 PG19 长文本数据集 ======================
def load_pg19_long_text():
print("正在流式加载 PG19 数据集...")
dataset = load_dataset("pg19", split=PG19_SPLIT, streaming=True, trust_remote_code=True)
for sample in dataset:
if len(sample["text"].split()) > 50000:
```

```
long_text = sample["text"]
break
print(f"PG19 原始文本长度：{len(long_text.split()):,} 词")
return long_text
# ===================== 3. 长文本记忆测试数据构造 =====================
def build_memory_test_data(full_text, context_len, needle=NEEDLE_KEY):
text_trunc = full_text[:context_len * 4]
insert_pos = np.random.randint(100, len(text_trunc) // 2)
test_text = text_trunc[:insert_pos] + needle + text_trunc[insert_pos:]
return test_text, needle
# ===================== 4. 文本转图像工具（适配视觉模型/MLP）
=====================
class Text2Image:
def __init__(self, img_size=(224, 224)):
self.img_size = img_size
self.transform = transforms.Compose([
transforms.Resize(img_size),
transforms.ToTensor(),
transforms.Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5])
])
def convert(self, text):
char_codes = [ord(c) % 256 for c in text]
pixel_num = self.img_size[0] * self.img_size[1]
if len(char_codes) < pixel_num:
char_codes += [0] * (pixel_num - len(char_codes))
else:
char_codes = char_codes[:pixel_num]
img_array = np.array(char_codes, dtype=np.uint8).reshape(self.img_size)
img = Image.fromarray(img_array).convert("RGB")
img_tensor = self.transform(img).unsqueeze(0).to(DEVICE)
return img_tensor
```

Assistant

```
 import torch
import torch.nn as nn
import numpy as np
import matplotlib.pyplot as plt
from tqdm import tqdm
from datasets import load_dataset
from PIL import Image
from transformers import AutoTokenizer, ViTImageProcessor,
ViTForImageClassification
from torchvision import models, transforms

# ===================== 1. Global Configuration =====================
if torch.cuda.is_available():
    DEVICE = "cuda"
elif torch.backends.mps.is_available():
    DEVICE = "mps"
else:
    DEVICE = "cpu"

print(f"Current compute device: {DEVICE}")
CONTEXT_LENGTHS = [1024, 4096, 8192, 16384]
NEEDLE_KEY = "Experiment Key: PG19_MEMORY_TEST_2026"
PG19_SPLIT = "test"
PLOT_SAVE_PATH = "memory_accuracy_curve.png"
plt.rcParams['font.sans-serif'] = ['DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False

# ===================== 2. Load PG19 Long Text Dataset =====================
def load_pg19_long_text():
    print("Streaming PG19 dataset...")
    dataset = load_dataset("pg19", split=PG19_SPLIT, streaming=True,
trust_remote_code=True)
    for sample in dataset:
        if len(sample["text"].split()) > 50000:
            long_text = sample["text"]
            break
    print(f"Original PG19 text length: {len(long_text.split()):,} words")
    return long_text

# ===================== 3. Construct Long Text Memory Test Data
=====================
def build_memory_test_data(full_text, context_len, needle=NEEDLE_KEY):
    text_trunc = full_text[:context_len * 4]
    insert_pos = np.random.randint(100, len(text_trunc) // 2)
    test_text = text_trunc[:insert_pos] + needle + text_trunc[insert_pos:]
    return test_text, needle

# ===================== 4. Text-to-Image Tool (Adapts to Vision Models/MLP)
```

```
=====================
class Text2Image:
    def __init__(self, img_size=(224, 224)):
        self.img_size = img_size
        self.transform = transforms.Compose([
            transforms.Resize(img_size),
            transforms.ToTensor(),
            transforms.Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5])
        ])

    def convert(self, text):
        char_codes = [ord(c) % 256 for c in text]
        pixel_num = self.img_size[0] * self.img_size[1]
        if len(char_codes) < pixel_num:
            char_codes += [0] * (pixel_num - len(char_codes))
        else:
            char_codes = char_codes[:pixel_num]
        img_array = np.array(char_codes, dtype=np.uint8).reshape(self.img_size)
        img = Image.fromarray(img_array).convert("RGB")
        img_tensor = self.transform(img).unsqueeze(0).to(DEVICE)
        return img_tensor
```

Turn 18

User

```
from transformers import AutoTokenizer, AutoModel
import torch
# ===================== 5. 各模型记忆测试器实现 =====================
import torch.nn.functional as F
EMBED_DIM = 768
HIDDEN_DIM = 1024
DROPOUT = 0.2
SEQ_MAX = 2048
class SimpleMambaBlock(nn.Module):
def __init__(self, d_model=EMBED_DIM, d_state=16):
super().__init__()
self.in_proj = nn.Linear(d_model, d_model * 2)
self.conv1d = nn.Conv1d(d_model, d_model, kernel_size=4, padding=3, groups=d_model)
self.x_proj = nn.Linear(d_model, d_state * 2 + d_model)
self.dt_proj = nn.Linear(d_model, d_model)
self.out_proj = nn.Linear(d_model, d_model)
def forward(self, x):
B, L, D = x.shape
xz = self.in_proj(x)
x_in, z = xz.chunk(2, dim=-1)
x_conv = x_in.transpose(1, 2)
x_conv = self.conv1d(x_conv)[:, :, :L].transpose(1, 2)
x_conv = F.silu(x_conv)
x_proj = self.x_proj(x_conv)
delta, A, B_val = torch.split(x_proj, [D, 16, 16], dim=-1)
delta = F.softplus(self.dt_proj(delta))
y = x_conv * delta
y = y * F.silu(z)
return self.out_proj(y)
```

```
class PureMambaMemoryTester:
def __init__(self, max_length=SEQ_MAX):
print("⏳ 正在初始化 Mamba 模型与分词器...")
self.tokenizer = AutoTokenizer.from_pretrained("state-spaces/mamba-130m-hf")
self.tokenizer.pad_token = self.tokenizer.eos_token
self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
self.model = AutoModel.from_pretrained(
"state-spaces/mamba-130m-hf",
torch_dtype=torch.bfloat16
).to(self.device).eval()
self.max_length = max_length
print(f"✅ 初始化完成！当前设备: {self.device}, 最大截断长度: {self.max_length}")
def test_memory(self, test_text, answer):
with torch.no_grad():
inputs = self.tokenizer(
test_text,
return_tensors="pt",
truncation=True,
max_length=self.max_length
).to(self.device)
outputs = self.model(
inputs.input_ids,
output_hidden_states=True,
return_dict=True
)
last_hidden = outputs.hidden_states[-1]
answer_inputs = self.tokenizer(
answer,
return_tensors="pt"
).to(self.device)
answer_outputs = self.model(
```

> answer_inputs.input_ids,
>
> output_hidden_states=True,
>
> return_dict=True
>
>)
>
> answer_hidden = answer_outputs.hidden_states[-1].mean(dim=1)
>
> text_hidden = last_hidden.mean(dim=1)
>
> similarity = torch.cosine_similarity(text_hidden, answer_hidden, dim=1).item()
>
> hit = similarity > 0.85
>
> return hit, similarity

🤖 Assistant

```
 from transformers import AutoTokenizer, AutoModel
import torch

# ===================== 5. Implementation of Memory Testers for Each Model
=====================
import torch.nn.functional as F

EMBED_DIM = 768
HIDDEN_DIM = 1024
DROPOUT = 0.2
SEQ_MAX = 2048

class SimpleMambaBlock(nn.Module):
    def __init__(self, d_model=EMBED_DIM, d_state=16):
        super().__init__()
        self.in_proj = nn.Linear(d_model, d_model * 2)
        self.conv1d = nn.Conv1d(d_model, d_model, kernel_size=4, padding=3,
groups=d_model)
        self.x_proj = nn.Linear(d_model, d_state * 2 + d_model)
        self.dt_proj = nn.Linear(d_model, d_model)
        self.out_proj = nn.Linear(d_model, d_model)

    def forward(self, x):
        B, L, D = x.shape
        xz = self.in_proj(x)
        x_in, z = xz.chunk(2, dim=-1)
        x_conv = x_in.transpose(1, 2)
        x_conv = self.conv1d(x_conv)[:, :, :L].transpose(1, 2)
        x_conv = F.silu(x_conv)
        x_proj = self.x_proj(x_conv)
        delta, A, B_val = torch.split(x_proj, [D, 16, 16], dim=-1)
        delta = F.softplus(self.dt_proj(delta))
        y = x_conv * delta
```

```
        y = y * F.silu(z)
        return self.out_proj(y)

class PureMambaMemoryTester:
    def __init__(self, max_length=SEQ_MAX):
        print("Initializing Mamba model and tokenizer...")

        self.tokenizer = AutoTokenizer.from_pretrained("state-spaces/mamba-130m-
hf")
        self.tokenizer.pad_token = self.tokenizer.eos_token

        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

        self.model = AutoModel.from_pretrained(
            "state-spaces/mamba-130m-hf",
            torch_dtype=torch.bfloat16
        ).to(self.device).eval()

        self.max_length = max_length
        print(f"Initialization complete! Current device: {self.device}, Max
truncation length: {self.max_length}")

    def test_memory(self, test_text, answer):
        with torch.no_grad():
            inputs = self.tokenizer(
                test_text,
                return_tensors="pt",
                truncation=True,
                max_length=self.max_length
            ).to(self.device)

            outputs = self.model(
                inputs.input_ids,
                output_hidden_states=True,
                return_dict=True
            )
            last_hidden = outputs.hidden_states[-1]

            answer_inputs = self.tokenizer(
                answer,
                return_tensors="pt"
            ).to(self.device)
            answer_outputs = self.model(
                answer_inputs.input_ids,
                output_hidden_states=True,
                return_dict=True
            )
            answer_hidden = answer_outputs.hidden_states[-1].mean(dim=1)

            text_hidden = last_hidden.mean(dim=1)
```

```
            similarity = torch.cosine_similarity(text_hidden, answer_hidden,
dim=1).item()

            hit = similarity > 0.85

        return hit, similarity
```

Turn 19

User

```
# ===================== 6. 实验主流程 + 结果可视化 =====================
def run_experiment():
pg19_text = load_pg19_long_text()
print("\n========== 初始化所有模型 ==========")
model_testers = {
"PyTorch_Mamba": PureMambaMemoryTester(),
"ViT": ViTMemoryTester(),
"ResNet50": ResNetMemoryTester(),
"MLP": MLPMemoryTester()
}
accuracy_results = {name: [] for name in model_testers.keys()}
num_trials = 20
print(f"\n========== 开始长文本记忆测试 (每个长度独立测试 {num_trials} 次) ==========")
for length in tqdm(CONTEXT_LENGTHS, desc="测试上下文长度"):
length_hits = {name: [] for name in model_testers.keys()}
for i in range(num_trials):
test_text, answer = build_memory_test_data(pg19_text, length)
for model_name, tester in model_testers.items():
hit = tester.test_memory(test_text, answer)
length_hits[model_name].append(hit)
for model_name in model_testers.keys():
avg_hit = np.mean(length_hits[model_name])
accuracy_results[model_name].append(avg_hit)
plot_accuracy_curve(accuracy_results, CONTEXT_LENGTHS)
print_report(accuracy_results, CONTEXT_LENGTHS)
# ===================== 7. 实验报告输出 =====================
def print_report(accuracy, context_lengths):
print("\n\n========== PG19 长文本记忆测试实验报告 ==========")
```

```
header = f"{'模型名':<10}" + "".join([f"{l:<10}" for l in context_lengths]) + f"{'平均准确率':<15}"
print(header)
print("-" * len(header))
for model_name, acc_list in accuracy.items():
avg_acc = np.mean(acc_list) * 100
acc_str = "".join([f"{acc*100:<10.1f}%" for acc in acc_list])
print(f"{model_name:<10}{acc_str}{avg_acc:<15.1f}%")
print("-" * len(header))
print(f"测试任务：大海捞针（长文本中随机插入唯一密钥检索）")
print(f"设备：{DEVICE} | 可视化结果已保存至：{PLOT_SAVE_PATH}")
# ===================== 8. 绘制准确率曲线 =====================
def plot_accuracy_curve(accuracy, context_lengths):
fig, ax = plt.subplots(figsize=(10, 6))
colors = ["#1f77b4", "#ff7f0e", "#2ca02c", "#d62728"]
markers = ["o", "s", "^", "x"]
model_names = list(accuracy.keys())
for i, name in enumerate(model_names):
ax.plot(
context_lengths,
accuracy[name],
label=name,
color=colors[i],
marker=markers[i],
linewidth=2,
markersize=8
)
ax.set_xlabel("Context Length (tokens)", fontsize=12, fontweight="bold")
ax.set_ylabel("Cosine Similarity (Feature Match)", fontsize=12, fontweight="bold")
ax.set_title("PG19 Long Text: Feature Similarity vs Context Length", fontsize=14,
fontweight="bold", pad=20)
ax.set_ylim(-0.2, 1.1)
ax.grid(True, alpha=0.3, linestyle="-")
```

ax.legend(loc="upper right", fontsize=10)

plt.tight_layout()

plt.savefig(PLOT_SAVE_PATH, dpi=300, bbox_inches="tight")

plt.close()

========== 主函数执行 ==========

if __name__ == "__main__":

torch.no_grad()

run_experiment()

Assistant

```
 import torch
import numpy as np
import matplotlib.pyplot as plt
from tqdm import tqdm

# ========== 6. Main Experiment Process + Result Visualization
==========
def run_experiment():
    pg19_text = load_pg19_long_text()

    print("\n========== Initialize All Models ==========")
    model_testers = {
        "PyTorch_Mamba": PureMambaMemoryTester(),
        "ViT": ViTMemoryTester(),
        "ResNet50": ResNetMemoryTester(),
        "MLP": MLPMemoryTester()
    }

    accuracy_results = {name: [] for name in model_testers.keys()}

    num_trials = 20
    print(f"\n========== Start Long Text Memory Test ({num_trials} independent
trials per length) ==========")

    for length in tqdm(CONTEXT_LENGTHS, desc="Testing Context Length"):
        length_hits = {name: [] for name in model_testers.keys()}

        for i in range(num_trials):
            test_text, answer = build_memory_test_data(pg19_text, length)

            for model_name, tester in model_testers.items():
                hit = tester.test_memory(test_text, answer)
                length_hits[model_name].append(hit)

        for model_name in model_testers.keys():
```

```
            avg_hit = np.mean(length_hits[model_name])
            accuracy_results[model_name].append(avg_hit)

    plot_accuracy_curve(accuracy_results, CONTEXT_LENGTHS)

    print_report(accuracy_results, CONTEXT_LENGTHS)

# ===================== 7. Experiment Report Output =====================
def print_report(accuracy, context_lengths):
    print("\n\n========== PG19 Long Text Memory Test Experiment Report ==========")
    header = f"{'Model':<12}" + "".join([f"{l:<10}" for l in context_lengths]) + f"
{'Avg Acc':<15}"
    print(header)
    print("-" * len(header))
    for model_name, acc_list in accuracy.items():
        avg_acc = np.mean(acc_list) * 100
        acc_str = "".join([f"{acc*100:<10.1f}%" for acc in acc_list])
        print(f"{model_name:<12}{acc_str}{avg_acc:<15.1f}%")
    print("-" * len(header))
    print(f"Test Task: Needle in a Haystack (Retrieval of a unique key randomly
inserted in long text)")
    print(f"Device: {DEVICE} | Visualization results saved to: {PLOT_SAVE_PATH}")

# ===================== 8. Plot Accuracy Curve =====================
def plot_accuracy_curve(accuracy, context_lengths):
    fig, ax = plt.subplots(figsize=(10, 6))
    colors = ["#1f77b4", "#ff7f0e", "#2ca02c", "#d62728"]
    markers = ["o", "s", "^", "x"]
    model_names = list(accuracy.keys())
    for i, name in enumerate(model_names):
        ax.plot(
            context_lengths,
            accuracy[name],
            label=name,
            color=colors[i],
            marker=markers[i],
            linewidth=2,
            markersize=8
        )
    ax.set_xlabel("Context Length (tokens)", fontsize=12, fontweight="bold")
    ax.set_ylabel("Cosine Similarity (Feature Match)", fontsize=12,
fontweight="bold")
    ax.set_title("PG19 Long Text: Feature Similarity vs Context Length",
fontsize=14, fontweight="bold", pad=20)
    ax.set_ylim(-0.2, 1.1)
    ax.grid(True, alpha=0.3, linestyle="-")
    ax.legend(loc="upper right", fontsize=10)
    plt.tight_layout()
    plt.savefig(PLOT_SAVE_PATH, dpi=300, bbox_inches="tight")
    plt.close()
```

```
# ===================== Main Function Execution =====================
if __name__ == "__main__":
    torch.no_grad()
    run_experiment()
```

Turn 20

User

生成一份
清
晰
的README文件，其中应有设置说明、依赖项以及运行命令。
请
对
代
码
库
进
行
整
理
，
以
便
读
者
能
够
轻
松
找
到
主

要的脚本和输出结果。说明数据集是如何获取或准备的。

- 说明报告中

的核心结果是由哪个脚本或笔记本重现的。

- 使用清晰的文件名，避免

> 不
> 必
> 要
> 的
> 杂
> 乱
> 。

🤖 **Assistant**

这是一份为您量身定制的标准化 `README.md` 文件。为了满足您对“清晰”、“易读”和“避免杂乱”的要求，我对原有的文件结构进行了逻辑层面的重新规划，建议您在本地也按照此结构对文件和文件夹进行重命名和归档。

您可以直接将以下内容复制并保存为项目根目录下的 `README.md` 文件。

📌 多模态深度学习架构评估基准 (Multi-Modal Model Evaluation Benchmark)

本项目是一个跨模态的深度学习模型评估基准，旨在横向对比经典架构（MLP, CNN, ResNet）与前沿架构（ViT, Transformer, Mamba, KAN）在不同任务和极端场景下的性能表现。评估涵盖三个核心维度：**图像鲁棒性 (Image)**、**时间序列预测 (Time Series)** 以及 **长文本极限记忆 (Long Text)**。

📂 推荐的代码库结构

为了保持代码库的整洁，建议按照以下目录结构整理您的文件。原有的三个 Jupyter Notebook 已被重新命名，以直观反映其对应任务。

```
├── README.md                          # 本说明文件
├── requirements.txt                   # 核心依赖清单
├── notebooks/                         # 核心执行脚本 (由原文件重命名)
│   ├── 1_Image_Robustness.ipynb       # 原 Image.ipynb (图像抗干扰与鲁棒性测试)
│   ├── 2_TimeSeries_Forecasting.ipynb # 原 Time Sequence.ipynb (金融时序预测与KAN测试)
│   └── 3_LongText_Memory.ipynb        # 原 Long Text Data.ipynb (PG19“大海捞针”记忆测试)
├── outputs/                           # 存放脚本运行生成的图表和报告 (自动生成)
│   ├── image_results/                 # 如: comprehensive_robustness_analysis.png
│   ├── time_series_results/           # 如: 预测折线图、MSE指标
```

```
│   └── long_text_results/          # 如: memory_accuracy_curve.png
└── saved_models/                   # 存放训练好的模型权重文件 (.pth)
```

⚙️ 设置说明与依赖项 (Setup & Dependencies)

本项目依赖于 Python 3.9+。由于涵盖了视觉、时序和状态空间模型 (Mamba)，需要安装特定的环境库。

1. 创建并激活虚拟环境 (推荐):

```
conda create -n multi-modal-eval python=3.10 -y
conda activate multi-modal-eval
```

2. 安装核心依赖包: 您可以创建一个 `requirements.txt` 文件，或直接运行以下命令：

```
# 基础深度学习与数据处理库
pip install torch torchvision torchaudio numpy pandas matplotlib scikit-learn tqdm jupyter

# 图像与文本大模型相关库
pip install transformers datasets pillow

# 时序金融数据获取
pip install yfinance

# 前沿架构支持 (Mamba & KAN)
pip install causal-conv1d>=1.2.0 mamba-ssm
pip install efficient-kan
```

(注：Mamba 的安装强烈建议在具备 NVIDIA GPU 且安装了 CUDA Toolkit 的 Linux/WSL 环境下进行)

🗄️ 数据集获取与准备 (Dataset Preparation)

本项目的优势在于完全自动化的数据准备流程，无需手动下载或清洗庞大的数据文件：

1. **图像数据集 (CIFAR-10):** - 运行 `1_Image_Robustness.ipynb` 时， `torchvision.datasets` 会自动下载 CIFAR-10 到本地 `./data` 目录，并自动应用自定义的干扰转换（椒盐噪声、空间遮挡、素描风格化）。
2. **时序数据集 (Yahoo Finance):** - 运行 `2_TimeSeries_Forecasting.ipynb` 时， `yfinance` 库会通过 API 实时拉取标普 500指数 (`^GSPC`) 过去 10 年的真实历史交易数据，并在内存中完成滑动窗口特征工程（构建 MA5, MA20 等特征）。
3. **长文本数据集 (PG19):** - 运行 `3_LongText_Memory.ipynb` 时，HuggingFace 的 `datasets` 库会以流式 **(streaming)** 方式加载 PG19 书籍数据集，自动截取长度超过 50,000 词的文

本块进行“大海捞针”实验，不占用大量本地硬盘空间。

🚀 运行命令与核心结果重现 (Reproducing Results)

启动 Jupyter 环境：

```
jupyter notebook
```

在浏览器中打开 `notebooks/` 文件夹，按照以下顺序或您的研究需求依次运行：

实验一：图像抗干扰与解释性测试

- **运行脚本:** `notebooks/1_Image_Robustness.ipynb`
- **核心目标:** 测试 MLP, CNN, ViT 等轻量级模型（~1.7M参数）在四种场景（纯净、噪声、遮挡、素描）下的表现。
- **重现核心结果:**
  1. 运行结束后，将输出字典文件 `evaluation_results.json` （包含 Acc, F1, ECE校准误差）。
  2. 自动生成多维图表 **`comprehensive_robustness_analysis.png`** （条形图、雷达图、效率散点图）。
  3. 自动生成特征注意力热力图矩阵 **`xai_saliency_all_scenarios.png`** 。

实验二：金融时间序列趋势预测

- **运行脚本:** `notebooks/2_TimeSeries_Forecasting.ipynb`
- **核心目标:** 对比 TS_MLP, TS_CNN, TS_Transformer, TS_Mamba 以及 TS_KAN 在时序数据上的收敛能力。
- **重现核心结果:**
  1. 脚本会输出精确的模型参数量对比表格。
  2. 自动计算各模型在测试集上的 **均方误差 (MSE)** 和 **方向准确率 (DA%)**。

实验三：极限长文本“大海捞针”记忆测试

- **运行脚本:** `notebooks/3_LongText_Memory.ipynb`
- **核心目标:** 利用文本像素化策略统一输入维度，测试 PureMamba, ViT, ResNet50, MLP 在最高 16,384 长度跨度下的特征检索引擎表现。
- **重现核心结果:**
  1. 控制台将输出详尽的“PG19长文本记忆测试实验报告”（包含具体命中率百分比）。

> 2. 自动在本地生成记忆衰减曲线图 `memory_accuracy_curve.png` 。

Turn 21

👤 User

> 删除requirements

🤖 Assistant

没问题，我已经为您移除了环境设置与依赖项（requirements）的相关部分，并同步更新了目录树结构。

以下是精简后的 `README.md` 文件：

📌 多模态深度学习架构评估基准 (Multi-Modal Model Evaluation Benchmark)

本项目是一个跨模态的深度学习模型评估基准，旨在横向对比经典架构（MLP, CNN, ResNet）与前沿架构（ViT, Transformer, Mamba, KAN）在不同任务和极端场景下的性能表现。评估涵盖三个核心维度：**图像鲁棒性 (Image)**、**时间序列预测 (Time Series)** 以及 **长文本极限记忆 (Long Text)**。

📂 推荐的代码库结构

为了保持代码库的整洁，建议按照以下目录结构整理您的文件。原有的三个 Jupyter Notebook 已被重新命名，以直观反映其对应任务。

```
├── README.md                          # 本说明文件
├── notebooks/                         # 核心执行脚本 (由原文件重命名)
│   ├── 1_Image_Robustness.ipynb       # 原 Image.ipynb (图像抗干扰与鲁棒性测试)
│   ├── 2_TimeSeries_Forecasting.ipynb # 原 Time Sequence.ipynb (金融时序预测与KAN测试)
│   └── 3_LongText_Memory.ipynb        # 原 Long Text Data.ipynb (PG19"大海捞针"记忆测试)
├── outputs/                           # 存放脚本运行生成的图表和报告 (自动生成)
│   ├── image_results/                 # 如: comprehensive_robustness_analysis.png
│   ├── time_series_results/           # 如: 预测折线图、MSE指标
│   └── long_text_results/             # 如: memory_accuracy_curve.png
└── saved_models/                      # 存放训练好的模型权重文件 (.pth)
```

🗄️ 数据集获取与准备 (Dataset Preparation)

本项目的优势在于完全自动化的数据准备流程，无需手动下载或清洗庞大的数据文件：

1. **图像数据集 (CIFAR-10):** - 运行 `1_Image_Robustness.ipynb` 时，`torchvision.datasets` 会自动下载 CIFAR-10 到本地 `./data` 目录，并自动应用自定义的干扰转换（椒盐噪声、空间遮挡、素描风格化）。
2. **时序数据集 (Yahoo Finance):** - 运行 `2_TimeSeries_Forecasting.ipynb` 时，`yfinance` 库会通过 API 实时拉取标普 500指数 (`^GSPC`) 过去 10 年的真实历史交易数据，并在内存中完成滑动窗口特征工程（构建 MA5, MA20 等特征）。
3. **长文本数据集 (PG19):** - 运行 `3_LongText_Memory.ipynb` 时，HuggingFace 的 `datasets` 库会以流式 **(streaming)** 方式加载 PG19 书籍数据集，自动截取长度超过 50,000 词的文本块进行“大海捞针”实验，不占用大量本地硬盘空间。

🚀 运行命令与核心结果重现 (Reproducing Results)

启动 Jupyter 环境：

```
jupyter notebook
```

在浏览器中打开 `notebooks/` 文件夹，按照以下顺序或您的研究需求依次运行：

实验一：图像抗干扰与解释性测试

- **运行脚本:** `notebooks/1_Image_Robustness.ipynb`
- **核心目标:** 测试 MLP, CNN, ViT 等轻量级模型（~1.7M参数）在四种场景（纯净、噪声、遮挡、素描）下的表现。
- **重现核心结果:**
  1. 运行结束后，将输出字典文件 `evaluation_results.json` （包含 Acc, F1, ECE校准误差）。
  2. 自动生成多维图表 **`comprehensive_robustness_analysis.png`** （条形图、雷达图、效率散点图）。
  3. 自动生成特征注意力热力图矩阵 **`xai_saliency_all_scenarios.png`** 。

实验二：金融时间序列趋势预测

- **运行脚本:** `notebooks/2_TimeSeries_Forecasting.ipynb`
- **核心目标:** 对比 TS_MLP, TS_CNN, TS_Transformer, TS_Mamba 以及 TS_KAN 在时序数据上的收敛能力。
- **重现核心结果:**
  1. 脚本会输出精确的模型参数量对比表格。
  2. 自动计算各模型在测试集上的 **均方误差 (MSE)** 和 **方向准确率 (DA%)**。

> ### 实验三：极限长文本“大海捞针”记忆测试
>
> - **运行脚本:** `notebooks/3_LongText_Memory.ipynb`
> - **核心目标:** 利用文本像素化策略统一输入维度，测试 PureMamba, ViT, ResNet50, MLP 在最高 16,384 长度跨度下的特征检索引擎表现。
> - **重现核心结果:**
> 1. 控制台将输出详尽的“PG19长文本记忆测试实验报告”（包含具体命中率百分比）。
> 2. 自动在本地生成记忆衰减曲线图 **`memory_accuracy_curve.png`** 。

Turn 22

👤 User

> 现在是一份要提交的readme文档，我希望你按照markdown格式整理出来，不需要任何建议性文字

🤖 Assistant

> ```
> # 📌 多模态深度学习架构评估基准 (Multi-Modal Model Evaluation Benchmark)
>
> 本项目是一个跨模态的深度学习模型评估基准，旨在横向对比经典架构（MLP, CNN, ResNet）与前沿架构（ViT, Transformer, Mamba, KAN）在不同任务和极端场景下的性能表现。评估涵盖三个核心维度：**图像鲁棒性 (Image)**、**时间序列预测 (Time Series)** 以及 **长文本极限记忆 (Long Text)**。
>
> ## 📂 代码库结构
>
> ```text
> ├── README.md                          # 本说明文件
> ├── notebooks/                         # 核心执行脚本
> │   ├── 1_Image_Robustness.ipynb       # 图像抗干扰与鲁棒性测试
> │   ├── 2_TimeSeries_Forecasting.ipynb # 金融时序预测与KAN测试
> │   └── 3_LongText_Memory.ipynb        # PG19"大海捞针"记忆测试
> ├── outputs/                           # 存放脚本运行生成的图表和报告 (自动生成)
> │   ├── image_results/                 # 如: comprehensive_robustness_analysis.png
> │   ├── time_series_results/           # 如: 预测折线图、MSE指标
> │   └── long_text_results/             # 如: memory_accuracy_curve.png
> └── saved_models/                      # 存放训练好的模型权重文件 (.pth)
>
> # 🗄️ 数据集获取与准备 (Dataset Preparation)
>
> 本项目的优势在于完全自动化的数据准备流程，无需手动下载或清洗庞大的数据文件：
>
> 1. **图像数据集 (CIFAR-10):** - 运行 `1_Image_Robustness.ipynb` 时，`torchvision.datasets` 会自动下载 CIFAR-10 到本地 `./data` 目录，并自动应用自定义的干扰转换（椒盐噪声、空间遮挡、素描风格化）。
> 2. **时序数据集 (Yahoo Finance):** - 运行 `2_TimeSeries_Forecasting.ipynb` 时，`yfinance` 库会通过 API 实时拉取标普 500指数 ( `^GSPC` ) 过去 10 年的真实历史交易数据，并在内存中完成滑动窗口特征工程（构建 MA5, MA20 等特征）。
> 3. **长文本数据集 (PG19):** - 运行 `3_LongText_Memory.ipynb` 时，HuggingFace 的 `datasets` 库会以流式 **(streaming)** 方式加载 PG19 书籍数据集，自动截取长度超过 50,000 词的文本块进行"大海捞针"实验，不占用大量本地硬盘空间。
>
> # 🚀 运行命令与核心结果重现 (Reproducing Results)
> ```

启动 Jupyter 环境：

```
jupyter notebook
```

在浏览器中打开 `notebooks/` 文件夹，按照以下顺序或您的研究需求依次运行：

实验一：图像抗干扰与解释性测试

- **运行脚本:** `notebooks/1_Image_Robustness.ipynb`
- **核心目标:** 测试 MLP, CNN, ViT 等轻量级模型（~1.7M参数）在四种场景（纯净、噪声、遮挡、素描）下的表现。
- **重现核心结果:**
  1. 运行结束后，将输出字典文件 `evaluation_results.json` （包含 Acc, F1, ECE校准误差）。
  2. 自动生成多维图表 **`comprehensive_robustness_analysis.png`** （条形图、雷达图、效率散点图）。
  3. 自动生成特征注意力热力图矩阵 **`xai_saliency_all_scenarios.png`** 。

实验二：金融时间序列趋势预测

- **运行脚本:** `notebooks/2_TimeSeries_Forecasting.ipynb`
- **核心目标:** 对比 TS_MLP, TS_CNN, TS_Transformer, TS_Mamba 以及 TS_KAN 在时序数据上的收敛能力。
- **重现核心结果:**
  1. 脚本会输出精确的模型参数量对比表格。
  2. 自动计算各模型在测试集上的 **均方误差 (MSE)** 和 **方向准确率 (DA%)**。

实验三：极限长文本“大海捞针”记忆测试

- **运行脚本:** `notebooks/3_LongText_Memory.ipynb`
- **核心目标:** 利用文本像素化策略统一输入维度，测试 PureMamba, ViT, ResNet50, MLP 在最高 16,384 长度跨度下的特征检索引擎表现。
- **重现核心结果:**
  1. 控制台将输出详尽的“PG19长文本记忆测试实验报告”（包含具体命中率百分比）。
  2. 自动在本地生成记忆衰减曲线图 **`memory_accuracy_curve.png`** 。

Turn 23

👤 User

> 本项目是一个跨模态的深度学习模型评估基准，旨在横向对比经典架构（MLP, CNN, ResNet）与前沿架构（ViT, Transformer, Mamba, KAN）在不同任务和极端场景下的性能表现。评估涵盖三个核心维度：图像鲁棒性 (Image)、时间序列预测 以及 长文本极限记忆。转换成英文

🤖 Assistant

> This project is a cross-modal deep learning model evaluation benchmark designed to horizontally compare the performance of classic architectures (MLP, CNN, ResNet) and cutting-edge architectures (ViT, Transformer, Mamba, KAN) across various tasks and extreme scenarios. The evaluation covers three core dimensions: **Image Robustness**, **Time Series Forecasting**, and **Extreme Long Text Memory**.

Turn 24

👤 User

> 本项目的优势在于完全自动化的数据准备流程，无需手动下载或清洗庞大的数据文件：
>
> 1. 图像数据集 (CIFAR-10): - 运行 `1_Image_Robustness.ipynb` 时，`torchvision.datasets` 会自动下载 CIFAR-10 到本地 `./data` 目录，并自动应用自定义的干扰转换（椒盐噪声、空间遮挡、素描风格化）。
>
> 2. 时序数据集 (Yahoo Finance): - 运行 `2_TimeSeries_Forecasting.ipynb` 时，`yfinance` 库会通过 API 实时拉取标普 500指数 (`^GSPC`) 过去 10 年的真实历史交易数据，并在内存中完成滑动窗口特征工程（构建 MA5, MA20 等特征）。
>
> 3. 长文本数据集 (PG19): - 运行 `3_LongText_Memory.ipynb` 时，HuggingFace 的 `datasets` 库会以流式 (streaming) 方式加载 PG19 书籍数据集，自动截取长度超过 50,000 词的文本块进行“大海捞针”实验，不占用大量本地硬盘空间。转换成英文

🤖 Assistant

> The advantage of this project lies in its fully automated data preparation process, eliminating the need to manually download or clean massive data files:
>
> 1. **Image Dataset (CIFAR-10):** When running `1_Image_Robustness.ipynb`, `torchvision.datasets` automatically downloads CIFAR-10 to the local `./data` directory and automatically applies custom perturbation transformations (salt-and-pepper noise, spatial occlusion, and sketch stylization).
> 2. **Time Series Dataset (Yahoo Finance):** When running `2_TimeSeries_Forecasting.ipynb`, the `yfinance` library fetches real historical trading data for the S&P 500 index (`^GSPC`) from the past 10 years in real-time via API, and completes sliding window feature engineering in-memory (constructing features such as MA5 and MA20).
> 3. **Long Text Dataset (PG19):** When running `3_LongText_Memory.ipynb`, HuggingFace's `datasets` library loads the PG19 book dataset in a streaming manner, automatically extracting text blocks exceeding 50,000 words for the "needle in a haystack" experiment without taking up massive amounts of local hard drive space.

Turn 25

👤 User

在浏览器中打开 `notebooks/` 文件夹，按照以下顺序或您的研究需求依次运行：

实验一：图像抗干扰与解释性测试

* 运行脚本: `notebooks/1_Image_Robustness.ipynb`

* 核心目标: 测试 MLP, CNN, ViT 等轻量级模型（~1.7M参数）在四种场景（纯净、噪声、遮挡、素描）下的表现。

* 重现核心结果:

1. 运行结束后，将输出字典文件 `evaluation_results.json`（包含 Acc, F1, ECE校准误差）。

2. 自动生成多维图表 `comprehensive_robustness_analysis.png`（条形图、雷达图、效率散点图）。

3. 自动生成特征注意力热力图矩阵 `xai_saliency_all_scenarios.png`。

实验二：金融时间序列趋势预测

* 运行脚本: `notebooks/2_TimeSeries_Forecasting.ipynb`

* 核心目标: 对比 TS_MLP, TS_CNN, TS_Transformer, TS_Mamba 以及 TS_KAN 在时序数据上的收敛能力。

* 重现核心结果:

1. 脚本会输出精确的模型参数量对比表格。

2. 自动计算各模型在测试集上的 均方误差 (MSE) 和 方向准确率 (DA%)。

实验三：极限长文本“大海捞针”记忆测试

* 运行脚本: `notebooks/3_LongText_Memory.ipynb`

* 核心目标: 利用文本像素化策略统一输入维度，测试 PureMamba, ViT, ResNet50, MLP 在最高 16,384 长度跨度下的特征检索引擎表现。

* 重现核心结果:

1. 控制台将输出详尽的“PG19长文本记忆测试实验报告”（包含具体命中率百分比）。

2. 自动在本地生成记忆衰减曲线图 `memory_accuracy_curve.png`。

```转换成英文

### 🤖 Assistant

Open the `notebooks/` folder in your browser and run them in the following order or according to your research needs:

## Experiment 1: Image Robustness and Interpretability Testing
```

- **Run Script**: `notebooks/1_Image_Robustness.ipynb`
- **Core Objective**: Test the performance of lightweight models (~1.7M parameters) such as MLP, CNN, and ViT under four scenarios (clean, noise, occlusion, and sketch).
- **Reproduce Core Results**:
  1. After running, it will output a dictionary file `evaluation_results.json` (containing Acc, F1, and ECE calibration error).
  2. Automatically generate a multi-dimensional chart **`comprehensive_robustness_analysis.png`** (bar chart, radar chart, and efficiency scatter plot).
  3. Automatically generate a feature attention heatmap matrix **`xai_saliency_all_scenarios.png`**.

Experiment 2: Financial Time Series Trend Forecasting

- **Run Script**: `notebooks/2_TimeSeries_Forecasting.ipynb`
- **Core Objective**: Compare the convergence capabilities of TS_MLP, TS_CNN, TS_Transformer, TS_Mamba, and TS_KAN on time series data.
- **Reproduce Core Results**:
  1. The script will output a precise model parameter count comparison table.
  2. Automatically calculate the **Mean Squared Error (MSE)** and **Directional Accuracy (DA%)** for each model on the test set.

Experiment 3: Extreme Long Text "Needle in a Haystack" Memory Test

- **Run Script**: `notebooks/3_LongText_Memory.ipynb`
- **Core Objective**: Utilize a text pixelation strategy to unify input dimensions and test the feature retrieval engine performance of PureMamba, ViT, ResNet50, and MLP across length spans up to 16,384.
- **Reproduce Core Results**:
  1. The console will output a detailed "PG19 Long Text Memory Test Experiment Report" (including specific hit rate percentages).
  2. Automatically generate a memory decay curve chart **`memory_accuracy_curve.png`** locally.

Exported from Gemini Voyager (https://github.com/Nagi-ovo/gemini-voyager) • 25 conversation turns
Generated on April 23, 2026 at 12:44 PM